EventoraX by ORBIT-I

Technical Documentation

Every folder and file, and how the parts connect

Version 1.0 · 11 October 2026

Contents

EventoraX technical documentation
- Chapters
- Progress checklist
- Keeping it current

1. Architecture
- The big picture
- Multi-tenancy: how organizations are kept apart
- Request lifecycle (an organization endpoint)
- Response envelope
- Authentication and sessions
- Roles
- Organization lifecycle
- Files and storage
- Background work
- Emails
- Frontend shape

2. Repository layout
- public/
- docs/
- backend/
- src/ (frontend)

3. Backend foundation
- index.ts: the Express app
- Middleware (middleware/)
- Prisma (prisma/)
- Utilities (utils/)
- web/spa.ts: serving the built site

4. Backend API
- Mounting (index.ts)
- Auth (routes/auth-route.ts)
- Organization (routes/org-routes.ts, controllers/org-controller.ts)
- Team (routes/team-routes.ts, controllers/team-controller.ts)
- Events and categories (routes/events-routes.ts, controllers/events-controller.ts)
- Speakers, sponsors, sessions, reorder (routes/program-routes.ts, controllers/program-controller.ts)
- Registrations (routes/registrations-routes.ts, controllers/registrations-controller.ts)
- Uploads (routes/uploads-routes.ts)
- Certificates and verification (routes/certificates-routes.ts, controllers/certificates-controller.ts)
- Tickets and check-in (routes/tickets-routes.ts, controllers/tickets-controller.ts)
- ID cards (routes/idcards-routes.ts, controllers/idcards-controller.ts)
- Dashboard, analytics, activity, billing, support (routes/dashboard-routes.ts)
- Notifications (routes/notifications-routes.ts)
- Public pages (routes/public-routes.ts, controllers/public-controller.ts)
- API tokens and the REST API (routes/api-routes.ts)
- Superadmin (routes/superadmin-routes.ts, controllers/superadmin/*)
- Activity action codes

5. Services, PDFs and background jobs
- Services (services/)
- PDF engines (pdf/)
- Background jobs (jobs/)

6. Database
- Conventions
- Entity overview
- Models
- Migrations
- Useful queries (Prisma Studio or SQL)

7. Frontend core

Entry: main.tsx
Routes: App.tsx
Talking to the backend: lib/api.ts
Contexts (context/)
Hooks (hooks/)
Helpers (lib/)
Types (types/)
Styles: index.css, Tailwind

8. Frontend components

ui/ (primitives)
app/ (building blocks)
layout/ (dashboard frame)
Feature folders

9. Frontend pages

Public website (pages/*.tsx)
Organization public pages (pages/public/)
Authentication pages
Organization dashboard (pages/dashboard/)
Superadmin (pages/superadmin/)
Developer pages (pages/dev/)

10. Operations

Checks (run before every commit)
CI (.github/workflows/ci.yml)
Building and running in production
Recipes
Troubleshooting
Known limitations and ideas

EventoraX technical documentation

For developers joining the project. It explains every folder and file, top to bottom, and how the parts connect. Read the chapters in order the first time; later, jump to the area you're changing.

Conventions that every change must follow (envelope, validation, logging, page templates, copy style) are in `CLAUDE.md`. This documentation explains how the existing code works.

Chapters

#	Chapter	Covers
1	Architecture	the big picture, multi-tenancy, request lifecycle, auth, roles, org lifecycle, files, emails
2	Repository layout	every top-level file and folder, config files, scripts
3	Backend foundation	`index.ts`, middleware, Prisma clients and scripts, all of `utils/`, `web/spa.ts`
4	Backend API	every router and controller, endpoint by endpoint
5	Services, PDFs and jobs	`services/`, `pdf/`, `jobs/`
6	Database	every model in `schema.prisma`, migrations
7	Frontend core	`main.tsx`, `App.tsx`, contexts, `lib/`, `hooks/`, `types/`
8	Frontend components	`components/ui`, `app`, `layout`, feature folders
9	Frontend pages	every page, what it calls, how it's built
10	Operations	checks, CI, building, deploying, adding features, troubleshooting

Progress checklist

All chapters were written on 2026-10-11 and checked against the code at commit time. If you add a chapter later, add a box here, and tick it only once every name in it has been verified against the code.

- [x] 1. Architecture
- [x] 2. Repository layout
- [x] 3. Backend foundation
- [x] 4. Backend API
- [x] 5. Services, PDFs and jobs
- [x] 6. Database
- [x] 7. Frontend core
- [x] 8. Frontend components
- [x] 9. Frontend pages
- [x] 10. Operations

How to continue: read the files of the next chapter's area (paths are listed in chapter 2), write the chapter in the same style (tables of files with their exports and purpose, then the flows that connect them), verify every name against the code, tick the box, commit with `docs(technical): …`.

Keeping it current

When you add or change a file, update the chapter that lists it in the same commit. A doc that disagrees with the code is worse than none.

1. Architecture

The big picture

```
Browser ──────────────────────────────────────────────────────────────────────
  React single-page app (src/)  ── fetch /api/v1/... with "Authorization: Bearer <JWT>"
       │                          ── <img src="/uploads/...">
       ▼
Development: Vite dev server :5173 ──proxy /api, /uploads──▶ Express :5000
Production:  Express :5000 serves dist/ (the built app) AND the API, one process
       │
       ▼
Express app (backend/src/index.ts)
  helmet → cors → json → apiLimiter → /uploads static → /health → maintenanceGate → routers → JSON 404
  → web/spa.ts (built site + link-preview tags) → error handler
       │
       ├── routes/*        URL → middleware chain → controller
       ├── middleware/*    who are you (JWT), which organization, can you write, plan still valid?
       ├── controllers/*   validate (zod) → read/write DB → respond { data, error, meta }
       ├── services/*      reusable business logic (certificates, tickets, notifications, announcements)
       ├── pdf/*           pdfkit renderers (certificates, tickets, ID cards, reports)
       ├── jobs/*          in-process scheduler (expire plans, reminders, announcements, summaries)
       └── utils/*         small helpers (http envelope, mail, settings cache, storage, crypto…)
       │
       ▼
Prisma 7 (MariaDB adapter) ──▶ MySQL 8          Disk: uploads/ (public), private-files/ (certificate PDFs)
                                                SMTP: system setting → .env → console ("DEV EMAIL")
```

Multi-tenancy: how organizations are kept apart

Every organization's data lives in the same tables, separated by an `organizationId` column. Three layers make sure one organization never sees another's rows:

1. **The JWT** says who the user is; `requireAuth` reloads the user from the database on every request, so their `organizationId` and role are always current.
2. `checkOrgStatus` loads that organization (`req.org`, with its plan) and refuses suspended or expired ones.
3. **The scoped Prisma client** (`req.db`, from `prisma/scopedClient.ts`) automatically adds `organizationId = <the user's org>` to every query on an organization-owned model (reads, updates, deletes) and to every create. A controller that uses `req.db` cannot reach other organizations' rows, even by mistake.

The base client (`prisma` from `prisma/client.ts`) has no such filter. It is only used for platform tables (plans, settings, users during login), by the superadmin, by public pages (which look rows up by an unguessable id), and for raw SQL, where the code must filter `organizationId` itself.

Request lifecycle (an organization endpoint)

Example: `PATCH /api/v1/registrations/:id/status` from the Registrations page.

1. `src/lib/api.ts` sends the request with the JWT from `localStorage` / `sessionStorage`.
2. `index.ts`: security headers, CORS, JSON body (5 MB max), rate limit (600/min/IP), maintenance gate.
3. `routes/registrations-routes.ts`: `orgScoped` (= `requireAuth` → `checkOrgStatus()` → `attachScopedPrisma`) then `canWrite` (viewers get 403).
4. `controllers/registrations-controller.ts`: parses the body with zod (`validationFail` → 400 with `fieldErrors`), updates through `req.db`, writes an activity row with `logActivity`, triggers side effects (certificate auto-issue), and answers `ok(res, data)`.
5. The page receives `{ data, error: null }`; on error the client throws an `ApiError` that forms turn into messages under each field (`applyServerErrors`).

Response envelope

Every endpoint except `/api/v1/auth/*` answers:

```
{ "data": ..., "error": null, "meta": { "total": 120, "page": 1, "limit": 20 } }
{ "data": null, "error": { "code": "VALIDATION_ERROR", "message": "...", "fieldErrors": { "email": "..." } } }
```

Auth routes predate the envelope and return plain JSON (`{ token, user }` or `{ error, code, fieldErrors }`). The frontend client understands both.

Authentication and sessions

- Passwords: bcrypt. Login: `POST /auth/login` → JWT signed with `JWT_SECRET` (1 day, or 30 days with "Remember me").
- The JWT holds `userId` and `tokenVersion` . Changing the password or "log out everywhere" increments `tokenVersion` , which instantly invalidates every older token (checked in `requireAuth`).
- Lockout: 5 wrong passwords for one email within 15 minutes (counted in `login_attempts`) → refused for 15 minutes.
- Two-factor login (TOTP): after the password, users with 2FA get a 5-minute **challenge token** signed with a different key; `POST /auth/login/2fa` exchanges it plus a code for a real session.
- Email verification, password reset and invites use random tokens that are stored **only as SHA-256 hashes**.
- **Impersonation** ("log in as"): the superadmin gets a 1-hour JWT for an org admin with `imp = <superadmin id>` . `requireAuth` checks the superadmin is still active; `logActivity` marks every action with `impersonatedBy` . The frontend keeps it in `sessionStorage` (that tab only) and remembers the superadmin's own session to return to.
- **API tokens** (`evx_live_…` , Enterprise) are a separate auth path for `/api/v1/ext` , see Backend API.

Roles

Role	Scope	Can
`superAdmin`	platform (`organizationId = null`)	everything under `/superadmin` , impersonate org admins
`admin`	one organization	everything in the organization, including team, billing, settings, API tokens
`manager`	one organization	create and change event data (`canWrite`)
`viewer`	one organization	read only

Backend: `requireRole([...])` and the shortcuts in `middleware/orgChain.ts` . Frontend: `useCan()("write" | "manage")` hides buttons; `ProtectedRoute roles` and `writer(page)` in `App.tsx` guard routes. The backend is the real gate; the frontend only avoids showing things that would fail.

Organization lifecycle

```
sign up ──▶ trial (trial.days setting) ──payment confirmed──▶ active (+1 year)
   │                │                                            │
   │                └──── end date passes ──▶ expired ◀──────────┘
   │                                            │ payment confirmed / superadmin extends
   └── superadmin suspends (any state) ──▶ suspended ──reactivate──▶ natural status
```

- `subscriptionEndsAt` is the single end date for trial and paid time.
- Expiry happens lazily on each request (`checkOrgStatus`) and every 10 minutes (`expire-orgs` job).
- Expired organizations keep their data. `orgScopedAllowExpired` routes (overview, billing, settings, support, some uploads) keep working; everything else answers `402 ORG_EXPIRED` .
- "Natural status" (`utils/orgStatus.ts`): expired if the date has passed, else active if it has ever paid, else trial.

Files and storage

Kind	Where on disk	Served	Example
Public uploads (logos, signatures, speaker photos, sponsor logos, receipts)	`backend/uploads/orgs/<orgId>/<folder>/`	`/uploads/...` (static)	org logo
Private files (certificate PDFs)	`backend/private-files/orgs/<orgId>/certificates/`	only through API endpoints that check access	`GET /certificates/:id/pdf`, `GET /verify/:code/pdf`
Generated on demand (tickets, ID cards, reports)	not stored	streamed	ticket PDF, ZIP
Backups	`backups/` (repo root)	superadmin download endpoint	`.sql` dumps

Logos and signatures must be PNG or JPEG because pdfkit prints them on PDFs.

Background work

- **Fire-and-forget:** long work after a response (bulk emails, certificate batches) runs in `void (async () => …)()`; errors are only logged.
- **Scheduled jobs** (`jobs/`): an in-process scheduler started by `index.ts` after `listen`. Fine for one server; if the API is ever run as several processes, move the jobs to a single worker.

Emails

All system emails go through `renderEmail(type, vars)` (`utils/mail.ts`): the superadmin's edited template from the `email_templates` table, or the built-in default in `utils/emailTemplates.ts`, wrapped in a standard layout. The SMTP server comes from the `smtp` system setting, else `backend/.env`, else the email is printed in the terminal.

Frontend shape

- `main.tsx` creates a **data router** (`createBrowserRouter`, needed by `useBlocker` for unsaved-changes prompts) with a single catch-all route that renders the providers and `App.tsx`, which declares all routes with `<Routes>`.
- Every page is `React.lazy` (one JS file per page). Libraries are grouped into cached chunks in `vite.config.ts`.
- Data fetching: `useApi` (cancellable, reloadable) over `lib/api.ts`. No global store: auth and platform settings live in two React contexts; everything else is per page, with filters in the URL (`useUrlState`).
- UI: Tailwind + shadcn/ui (Radix) primitives in `components/ui`, app-level building blocks in `components/app` (tables, toolbars, states, dialogs), feature components in `components/<feature>`.

2. Repository layout

One repository, two Node projects: the **frontend** at the root and the **backend** in `backend/`. Each has its own `package.json`, `node_modules` and TypeScript config. They talk only over HTTP.

```
EventoraX-full/
├── .github/workflows/ci.yml   CI: frontend build + knip, backend typecheck + knip (push/PR to main, integration/v1)
├── backend/                   API server (see chapters 3–6)
├── docs/                      documentation
├── public/                    static files copied as-is into dist/
├── src/                       React app (see chapters 7–9)
├── CLAUDE.md                  project guide and agreed decisions (also read by Claude Code)
├── README.md                  what it is, how to run it
├── components.json            shadcn/ui CLI settings (where `npx shadcn add <x>` puts components)
├── eslint.config.js           ESLint flat config (TS, react-hooks, react-refresh). Not part of the checks yet.
├── index.html                 Vite entry HTML: default meta/SEO/og tags, favicon, #root, loads src/main.tsx
├── knip.json                  unused files/exports/deps checker for the frontend
├── package.json               frontend dependencies and scripts (dev, build, lint, preview)
├── postcss.config.js          Tailwind + autoprefixer
├── tailwind.config.js         theme (brand purple scale, fonts, shadcn CSS variables), tailwindcss-animate
├── tsconfig.json              references tsconfig.app.json + tsconfig.node.json, `@/*` → `src/*`
├── tsconfig.app.json          strict TS for src/ (bundler resolution, noUnused*, erasableSyntaxOnly)
├── tsconfig.node.json         TS for vite.config.ts
└── vite.config.ts             dev server :5173 with proxy to :5000, `@` alias, manual chunks for libraries
```

Git-ignored (local only): `node_modules/`, `dist/` (built site), `backend/dist/` (compiled API), `backend/.env`, `backend/uploads/`, `backend/private-files/`, `backups/`.

`public/`

File	Used by
`favicon.svg`	default browser-tab icon (`index.html`); replaced at runtime if System settings has a favicon
`og-image.png`	default share image (`index.html`, `backend/src/web/spa.ts` for pages without their own image)
`images/logo-orbit.png`	ORBIT-I logo in the home page hero (`components/home/HeroSection.tsx`)

`docs/`

Path	Audience
`requirements.md`	the original specification (all phases)
`product-guide/`	users of every role, with screenshots
`handover.md`	team leads: placeholders to replace before launch
`manual-testing.md`	click-by-click test scripts per area
`technical/`	developers (this documentation)

`backend/`

```
backend/
├── .env.example        every environment variable with comments (copy to .env)
├── api-tests.http      ready-made requests for the VS Code "REST Client" extension
├── knip.json           unused-code checker config (fonts loaded by file path are whitelisted)
├── package.json        scripts: dev (tsx watch), build (tsc → dist/), start, seed:demo, reset-2fa
├── prisma.config.ts    Prisma 7 config: schema path, migrations path, seed command, DATABASE_URL
├── tsconfig.json       CommonJS, ES2020, strict + noUncheckedIndexedAccess, src → dist
├── prisma/
│   ├── schema.prisma   the database schema (chapter 6)
│   └── migrations/     SQL migrations, applied in order by `prisma migrate deploy|dev`
└── src/
    ├── index.ts        the Express app
    ├── controllers/    request handlers, one file per area (+ superadmin/)
    ├── jobs/           scheduler + job definitions
    ├── middleware/     auth, org status, scoped DB, roles, API tokens, maintenance
    ├── pdf/            pdfkit renderers and shared assets
    ├── prisma/         Prisma clients and seed / maintenance scripts
    ├── routes/         routers, one file per area
    ├── services/       shared business logic used by several controllers and jobs
    ├── utils/          small helpers
    └── web/spa.ts      serves the built frontend with link-preview tags
```

`backend/api-tests.http`

Open it in VS Code with the **REST Client** extension; a "Send Request" link appears above each request. Variables at the top (`@baseUrl` , `@token` , `@eventId` , `@apiToken` …) are filled in by hand after logging in. It covers auth, every organization area, public pages, the REST API and some superadmin calls. Keep it updated when adding endpoints: it is the quickest way to test the API without the UI.

Scripts in `backend/package.json`

Script	Runs
`npm run dev`	`tsx watch src/index.ts` (restarts on save)
`npm run build`	`tsc → backend/dist/`
`npm start`	`node dist/index.js` (production)
`npm run seed:demo -- --yes`	`src/prisma/seed-demo.ts` : wipes all organizations and builds the demo one (refuses in production)
`npm run reset-2fa -- <email>`	`src/prisma/reset-2fa.ts` : turns off two-factor login for one account
`npx prisma db seed`	`src/prisma/seed.ts` (configured in `prisma.config.ts`): plans, settings, superadmin

`src/` (frontend)

```
src/
├── main.tsx            entry: providers + data router
├── App.tsx             every route
├── index.css           Tailwind layers, CSS variables, fonts, utility classes
├── components/
│   ├── ui/             shadcn/ui primitives (+ our button and form fields)
│   ├── app/            app-wide building blocks (tables, toolbars, states, dialogs, pickers)
│   ├── layout/         dashboard shell: sidebar, top bar, banners, nav definitions
│   ├── home/           landing page sections
│   ├── public/         public site pieces (plans, legal layout, public shell, social links)
│   ├── auth/           AuthShell (layout of login/register pages) and RouteGuards (Protected/Guest routes)
│   ├── <feature>/      events, registrations, speakers, sponsors, sessions, team, settings, billing,
│   │                   certificates, tickets, idcards, superadmin
│   └── (root files)    Navbar, Footer, OceanCanvas (3D hero waves)
├── context/            AuthContext, PlatformContext
├── hooks/              useApi, useUrlState, useSelectedEvent, useUnsavedChanges, useVisibilityToggle, useNow
├── lib/                api client, formatting, statuses, permissions, validation, small helpers
├── pages/              one file per route: public pages at the root, dashboard/, superadmin/, public/, dev/
└── types/              TypeScript shapes of API responses, one file per area
```

205 files, about 28,000 lines. Details in chapters 7–9.

3. Backend foundation

The files every endpoint relies on: the app setup, middleware, Prisma clients, utilities and the web server for the built site. Paths are relative to `backend/src/`.

`index.ts`: the Express app

Order matters; this is the order in the file:

1. `dotenv/config` loads `backend/.env`.
2. `trust proxy = 1` so `req.ip` is the real client behind Hostinger/Nginx (rate limits and logs depend on it).
3. **helmet**: security headers with a Content Security Policy that allows images from `https:`, `data:`, `blob:` (org logos by URL, PDF previews), Google Fonts, and blob workers. HSTS and `upgrade-insecure-requests` only when `NODE_ENV=production` (they would break `http://localhost`).
4. **cors** for `FRONTEND_URL` (only matters if the site and API are on different origins).
5. `express.json({ limit: "5mb" })`.
6. `apiLimiter` on `/api`: 600 requests per minute per IP.
7. `/uploads` static files from `UPLOAD_ROOT`.
8. `GET /api/v1/health`: `SELECT 1` → `{ ok, db }` (used by monitoring and the Maintenance page).
9. `maintenanceGate` for everything under `/api/v1` below this point.
10. Public platform info (`/platform`, `/platform/website`), then every router (table in Backend API).
11. A JSON 404 for any other `/api` path.
12. `mountWebApp(app)`: serves `dist/` (see below).
13. The error handler: multer errors → 400; errors with a numeric `status` use it; everything else → 500 with a generic message. 5xx errors are printed and saved to `error_logs` (`recordError`).
14. `listen(PORT)` and `startScheduler()` (background jobs).

Unhandled promise rejections in Express 5 route handlers reach the error handler automatically, so controllers can `throw`.

Middleware (`middleware/`)

File	Exports	What it does
`auth.ts`	`requireAuth`, `AuthUser`	Reads `Authorization: Bearer <jwt>`, verifies it, **reloads the user** (must exist, be active, and have the same `tokenVersion`), checks an impersonating superadmin is still an active superadmin, sets `req.user = { userId, email, name, role, organizationId, impersonatorId? }`. 401 `NO_TOKEN` / `BAD_TOKEN` otherwise.
`checkOrgStatus.ts`	`checkOrgStatus({ allowExpired })`, `OrgWithPlan`	Loads `req.org` (with `plan`). 403 `NO_ORG` for users without an org (superadmin), 403 `ORG_SUSPENDED` (with the reason), flips trial/active orgs past `subscriptionEndsAt` to `expired`, then 402 `ORG_EXPIRED` unless `allowExpired`.
`scopedPrisma.ts`	`attachScopedPrisma`	Sets `req.db = getScopedPrisma(orgId)`.
`role-check.auth.ts`	`requireRole(roles)`	401 without `req.user`, 403 if the role isn't listed.
`orgChain.ts`	`orgScoped`, `orgScopedAllowExpired`, `canWrite`, `adminOnly`, `superAdminOnly`	The chains routes actually use. `orgScoped = [requireAuth, checkOrgStatus(), attachScopedPrisma]`. `canWrite` = admin or manager.
`apiToken.ts`	`requireApiToken`, `planHasApi`, `API_TOKEN_PREFIX`	Auth for `/api/v1/ext`: `Bearer evx_live_…` → SHA-256 lookup in `api_tokens`; refuses revoked/expired tokens, suspended/expired orgs and plans without `features.restApi`. Updates `lastUsedAt` at most once a minute. Sets `req.org`, `req.db`, `req.apiToken`.
`maintenance.ts`	`maintenanceGate`	When the `maintenance.enabled` setting is on, answers `503 MAINTENANCE` (with the configured message) except for `/auth`, `/health`, `/platform`, `/superadmin`, `/verify`, `/contact`, `/plans`, superadmins and impersonation sessions.

Typical route declarations:

```
router.get("/", orgScoped, list);                        // any member
router.post("/", orgScoped, canWrite, create);           // admin + manager
router.patch("/me", orgScopedAllowExpired, adminOnly, update); // admin, even after expiry
router.get("/tenants", superAdminOnly, listTenants);     // superadmin
```

The `Request` type is extended with `user`, `org`, `db` and `apiToken` through `declare global` blocks in these files, which is why controllers can write `req.db!` and `req.org!`.

Prisma (`prisma/`)

File	Purpose
`client.ts`	The one `PrismaClient`, using the **MariaDB driver adapter** (`@prisma/adapter-mariadb`, works with MySQL 8) built from `DB_*` env vars, pool of 5 connections, `allowPublicKeyRetrieval` for MySQL 8 auth without SSL. Default export `prisma`.
`scopedClient.ts`	`getScopedPrisma(orgId)`: a `$extends` query extension. For the models in `ORG_SCOPED_MODELS` (User, Invitation, Event, EventCategory, Registration, Speaker, Sponsor, Session, Certificate, Ticket, IdCard, Payment, ActivityLog, Notification, ApiToken) it adds `organizationId` to `where` (find*, count, aggregate, groupBy, update*, delete*), to `data` (create, createMany) and to `upsert`. **A new model with an `organizationId` column must be added to this set.**
`seed.ts`	`npx prisma db seed`: upserts the Pro and Enterprise plans (**overwriting** their price, limits and features on every run), the superadmin from `SEED_SUPERADMIN_*` (never overwrites) and default system settings (only missing keys).
`seed-demo.ts`	`npm run seed:demo -- --yes`: deletes every organization (and its uploads/private files) and builds the demo organization with realistic data and real certificate PDFs. Refuses without `--yes` and when `NODE_ENV=production`. Sends no email.
`reset-2fa.ts`	`npm run reset-2fa -- <email>`: clears the account's 2FA secret and recovery codes (for a superadmin who lost their phone).

`findUnique` with the scoped client: Prisma requires unique fields in `findUnique`'s `where`; adding `organizationId` works because Prisma 5+ accepts extra non-unique filters there.

Utilities (`utils/`)

HTTP and validation

File	Exports
`http.ts`	`ok(res, data, status?, meta?)`, `fail(res, status, code, message, { fieldErrors })`, `validationFail(res, zodError)` (400 `VALIDATION_ERROR`), `pagination(query, max=100)` → `{ page, limit, skip }`, `q(value)` (trimmed query string or `undefined`)
`validation.ts`	`fieldErrors(zodError)` → `{ "path.to.field": "first message" }`
`schemas.ts`	zod building blocks: `optionalText(max)`, `optionalUrl`, `optionalImage` (http(s) link or `/uploads/…`), plus `isOneOf`, `parseDate(value, endOfDay)`, `isUniqueViolation(err)` (Prisma P2002)
`password.ts`	`PASSWORD_REGEX` (8+ chars, uppercase, digit, special), `PASSWORD_RULE_MESSAGE`, `isStrongPassword`, `EMAIL_REGEX`
`csv.ts`	`csvCell` (quotes and neutralises `= + - @` formula injection), `toCsv(header, rows)` (UTF-8 BOM so Excel shows Urdu names)

Security

File	Exports
`jwt.ts`	`signAuthToken(payload, remember)` (1 day / 30 days), `verifyAuthToken`, `signImpersonationToken` (1 hour, `imp`), `signTwoFactorChallenge` / `verifyTwoFactorChallenge` (5 minutes, signed with `JWT_SECRET + ":two-factor-challenge"`, `purpose: "2fa"`, so it can never pass as a session). Throws if `JWT_SECRET` is shorter than 32 characters.
`tokens.ts`	`randomToken(bytes)` (hex), `hashToken` (SHA-256 hex). Email tokens and API tokens are stored only as hashes.
`secrets.ts`	`encryptSecret` / `decryptSecret` / `isEncrypted`: AES-256-GCM with a key = SHA-256 of `ENCRYPTION_KEY` (or `JWT_SECRET + ":eventorax-stored-secrets"`). Format `enc:v1:<iv>:<tag>:<data>` (base64). Plain values pass through `decryptSecret` unchanged (old data). Used for the SMTP password and TOTP secrets.
`totp.ts`	RFC 6238 without a library: `newTotpSecret` (160-bit base32), `verifyTotp(secret, code)` (±1 step, timing-safe), `otpauthUrl` (for the QR), `newRecoveryCodes(10)` (`xxxx-xxxx`, no look-alike characters), `normaliseRecoveryCode`

File	Exports
`rateLimits.ts`	express-rate-limit instances with the JSON 429 envelope: `loginLimiter` 60/15 min, `twoFactorLimiter` 15/15 min, `registerLimiter` 5/hour, `passwordLimiter` 5/15 min, `emailLinkLimiter` 10/hour, `inviteLimiter` 20/hour, `apiLimiter` 600/min. All per IP; kept generous because a campus shares one IP.

Data helpers

File	Exports
`activity.ts`	`logActivity(req, { action, entityType?, entityId?, metadata?, organizationId?, userId? })` : writes `activity_logs` with IP and user agent; adds `impersonatedBy` while impersonating; never throws. Action codes are `area.verb` (e.g. `registration.status`); the frontend turns them into sentences in `src/lib/activityText.ts` .
`codes.ts`	`newRefNo()` → `REG-XXXXXXXXXX` , `newTicketNo()` → `TKT-XXXXXXXXXX` , `newQrCode()` (32 hex chars inside the QR)
`settings.ts`	`getSetting(key, fallback)` with a 30-second in-memory cache, `setSetting(key, value)` (upsert + forget cache), `clearSettingsCache()`
`orgStatus.ts`	`expireOverdueOrgs()` (bulk trial/active → expired), `naturalStatus(orgId, endsAt)` , `DAY_MS`
`orgContacts.ts`	`orgAdminEmails(orgId)` : active admins + the org's own email, de-duplicated
`roleLimit.ts`	`seatLimitFor(org, role)` (plan's `maxAdmins` / `maxManagers` , `null` = unlimited, viewers unlimited), `seatsUsed(db, role)` (members + valid pending invites)
`months.ts`	`monthStart(n)` , `fillMonths(rows, count)` (fills missing months with 0 for charts)
`errorLog.ts`	`recordError(err, req, status)` → `error_logs` (never throws)

Files and email

File	Exports
`storage.ts`	Public uploads: `UPLOAD_ROOT` , `saveUpload(folder, buffer, mime)` → `/uploads/<folder>/<uuid>.<ext>` , `deleteUpload(publicPath)` , `deleteUploadFolder(folder)` . Private files: `PRIVATE_ROOT` , `savePrivate(key, buffer)` , `readPrivate(key)` (null if missing), `deletePrivateFolder(folder)` . All with path-traversal guards.
`upload.ts`	multer configs (memory storage): `imageUpload` (PNG/JPEG/WEBP, hard cap 20 MB), `csvUpload` (.csv, 2 MB); `checkImageSize` / `imageSizeGuard` apply the `uploads.maxMb` setting; `UploadError` (status 400)
`emailTemplates.ts`	The 10 editable templates (`TEMPLATE_DEFINITIONS` : type, name, description, where it's used, variables with samples, default subject and HTML). `fill(text, vars, html)` handles `{{var}}` (escaped), `{{{var}}}` (raw) and `{{button "Label" var}}` ; unknown variables become empty. `loadTemplate(type)` (DB row or default), `sampleVars` , `escapeHtml` .
`mail.ts`	Transport selection (setting `smtp` → `.env` → console), cached nodemailer transporter, `mailStatus()` , `verifyMailTransport()` , the shared `layout()` (purple header, platform name), `sendMail(to, subject, html, devLink?, attachments?)` , `renderEmail(type, vars, override?)` , and one function per email (`sendVerificationEmail` , `sendResetPasswordEmail` , `sendInviteEmail` , `sendWelcomeEmail` , `sendCertificateEmail` , `sendTicketEmail` , `sendRegistrationConfirmEmail` , `sendRenewalEmail` , `sendExpiryWarningEmail` , `sendAnnouncementEmail` , plus the non-template `sendAttendeeMessage` and `sendNotice`). Links use `FRONTEND_URL` .

Adding a new system email: add a type to the `EmailTemplateType` enum (schema + migration), add a definition to `TEMPLATE_DEFINITIONS` , add a `send…` function in `mail.ts` that calls `sendTemplate` . The superadmin editor picks it up automatically.

`web/spa.ts` : serving the built site

`mountWebApp(app)` does nothing if `dist/index.html` doesn't exist (development). Otherwise:

- Static files from `dist/` (`WEB_DIST` overrides the path). Hashed files in `/assets/` are cached for a year (`immutable`), other files for an hour.
- Any other GET outside `/api/` and `/uploads/` returns `index.html` (single-page app fallback). Paths that look like files (`.js` , `.png` …) and don't exist get a real 404, so an old chunk after a deploy fails cleanly (the frontend then reloads once, see `vite:preloadError` in `main.tsx`).
- **Link previews:** social apps don't run JavaScript, so for `/e/:id` (event), `/o/:slug` (organization) and `/verify/:code` (certificate) the server looks up the record and rewrites `<title>` , description, canonical, Open Graph and Twitter tags in the HTML (org logo as image, or `og-image.png`). Hidden/unknown records answer 404 with generic tags. Other pages get the SEO title/description from System settings.
- The template is re-read when `dist/index.html` changes, so a rebuild needs no restart.
- In production the public origin comes from `FRONTEND_URL` ; otherwise from the request's host.

4. Backend API

Every endpoint, grouped by router. All paths start with `/api/v1`. Paths are relative to `backend/src/`.

How routes and controllers are split: `routes/<area>-routes.ts` declares URLs and the middleware chain; `controllers/<area>-controller.ts` holds the handlers (zod schema at the top of each section, then the handler). The exception is `routes/auth-route.ts`, which contains its handlers inline (it predates the split).

Route order rule: fixed paths (`/stats`, `/export`, `/bulk`, `/zip`) are declared before `/:id`, otherwise Express would treat `stats` as an id.

Legend: *any* = any member of the organization (viewers included), *write* = admin + manager (`canWrite`), *admin* = organization admins, *expired ok* = still works when the plan has expired.

Mounting (`index.ts`)

Prefix	Router (file)	Auth
`/health`	inline	public
`/platform`, `/platform/website`	`controllers/superadmin/settings-controller.ts`	public
`/plans`	`controllers/superadmin/plans-controller.ts` → `publicPlans`	public
`/auth`	`routes/auth-route.ts`	mixed
`/org`	`routes/org-routes.ts`	member, expired ok
`/team`	`routes/team-routes.ts`	member
`/events`, `/categories`	`routes/events-routes.ts`	member
`/speakers`, `/sponsors`, `/sessions`, `/reorder`	`routes/program-routes.ts`	member
`/registrations`	`routes/registrations-routes.ts`	member
`/uploads`	`routes/uploads-routes.ts`	member, expired ok (some kinds)
`/certificates`, `/verify`	`routes/certificates-routes.ts`	member / public
`/tickets`	`routes/tickets-routes.ts`	member
`/idcards`	`routes/idcards-routes.ts`	member
`/dashboard`, `/analytics`, `/activity`, `/billing`, `/support`, `/contact`	`routes/dashboard-routes.ts`	mixed
`/notifications`	`routes/notifications-routes.ts`	member, expired ok
`/public`	`routes/public-routes.ts`	public
`/api-tokens`, `/ext`	`routes/api-routes.ts`	admin (Enterprise) / API token
`/superadmin`	`routes/superadmin-routes.ts`	superadmin

Auth (`routes/auth-route.ts`)

Plain JSON responses (no envelope): success `{ token, user, … }` or `{ message }`; errors `{ error, code?, fieldErrors? }`. `publicUser()` is the shape returned for a user: `id, name, email, phone, role, organizationId, emailVerified, twoFactorEnabled`.

Endpoint	Limiter	What it does
`POST /register` `{ fullName, organizationName, email, phone, password }`	5/h	Validates (strong password, Pakistani phone via libphonenumber), creates in one transaction the organization (`status: trial`, plan Pro, `subscriptionEndsAt = now + trial.days`, unique slug from the name) + the admin user + an email-verification token (24 h). Sends verification and welcome emails. Returns a session token.
`GET /verify-email?token=`	10/h	Marks the email verified (token hash lookup, single use).

Endpoint	Limiter	What it does
`POST /resend-verification` (logged in)	10/h	New verification token + email.
`POST /login { email, password, remember? }`	60/15 min	Lockout check (5 failures since the later of 15 min ago and the last success), bcrypt compare against a dummy hash when the email doesn't exist (equal timing), disabled users and suspended orgs refused (with reason). With 2FA: `{ twoFactorRequired, challengeToken }`. Otherwise records the attempt, sets `lastLoginAt`, logs `auth.login`, returns `{ token, user, organizationStatus }`.
`POST /login/2fa { challengeToken, code }`	15/15 min	Verifies the challenge, then a TOTP code or an unused recovery code (which is then removed). Failures count toward the lockout. Returns the session.
`POST /2fa/setup` (logged in)		New secret (encrypted, stored but not active) + `otpauth://` URL for the QR.
`POST /2fa/enable { code }`	15/15 min	Confirms a code, sets `twoFactorEnabledAt`, returns 10 recovery codes (stored as hashes).
`POST /2fa/disable { password }`, `POST /2fa/recovery-codes { password }`	5/15 min	Turn off / regenerate codes. Blocked while impersonating.
`POST /forgot-password { email }`	5/15 min	Always answers the same message; if the account exists, a 15-minute reset token is emailed.
`POST /reset-password { token, newPassword }`	5/15 min	Sets the password, bumps `tokenVersion` (logs out everywhere), deletes the user's reset tokens.
`GET /me`		Current user + organization (with plan) + `impersonator` if impersonating.
`PATCH /me { name?, phone? }`		Edit own profile.
`POST /change-password { currentPassword, newPassword }`	5/15 min	Bumps `tokenVersion` and returns a fresh token so this tab stays logged in. Blocked while impersonating.
`POST /logout-all`		Bumps `tokenVersion`.
`GET /invite/:token`	20/h	Public: who invited you to which organization and role.
`POST /accept-invite { token, name, password, phone? }`	20/h	Creates the user in the inviting org with the invited role (the seat was already reserved: pending invites count toward the limit), marks the invite accepted, returns a session.

Organization (`routes/org-routes.ts` , `controllers/org-controller.ts`)

Chain: `requireAuth` + `checkOrgStatus({ allowExpired: true })` (no scoped client; the controller uses `req.org`).

Endpoint	Who	What
`GET /org/me`	any	The organization with its plan.
`PATCH /org/me`	admin	Any subset of: name, email, phone, logoUrl, signatureUrl, primaryColor/accentColor (`#rrggbb`), signatoryName/Title, whiteLabelName, customDomain (hostname, unique), `notificationPrefs { newRegistration, attendance, certificate }`. Empty string clears a field. Replaced logo/signature files are deleted.
`POST /org/me/delete-data { confirm, password }`	admin	`confirm` must equal the org name exactly and the password must match. Deletes every event (cascades to registrations, tickets, certificates, speakers, sponsors, sessions, categories) and the speaker/sponsor uploads and private files.
`POST /org/me/close { confirm, password }`	admin	Deletes the organization (cascades to users and everything else) and all its files. Logged at platform level.

Logo and signature are uploaded first with `POST /uploads/image?kind=logo|signature`, then saved with `PATCH /org/me`.

Team (`routes/team-routes.ts` , `controllers/team-controller.ts`)

Endpoint	Who	What
`GET /team`	any	Members (name, email, role, status, last login) + pending invites + seat usage per role.
`POST /team/invite { email, role }`	admin	Refuses emails that already have an account (an email belongs to one account and one organization) or a pending invite; checks seat limits (`seatsUsed` counts members + valid invites); upserts the invitation with a hashed token valid 7 days; emails the link.
`POST /team/invites/:inviteId/resend`	admin	New token and 7 more days; the old link stops working.
`DELETE /team/invites/:inviteId`	admin	Cancels.
`PATCH /team/:userId/role { role }`	admin	Not your own role; keeps at least one admin; checks the seat limit of the new role. Takes effect immediately (role is read from the DB per request).
`DELETE /team/:userId`	admin	Not yourself; keeps one admin; reassigns invites they sent to you; deletes the user.

Events and categories (`routes/events-routes.ts` , `controllers/events-controller.ts`)

Statuses `DRAFT` | `PUBLISHED` | `ONGOING` | `COMPLETED` | `ARCHIVED` (UI: Draft, Upcoming, Active, Completed, Archived). Modes `ONLINE` | `OFFLINE` | `HYBRID` (UI: Online, Physical, Hybrid).

Endpoint	Who	What
`GET /events?status=&mode=&search=&from=&to=&page=&limit=`	any	Paged list with registration counts. Default excludes archived.
`GET /events/stats`	any	Counts per status for the list's cards and tabs.
`POST /events`	write	Creates with categories (labels de-duplicated case-insensitively). Online/hybrid need a `meetingLink` .
`GET /events/:id`	any	Event + categories + attendance numbers.
`GET /events/:id/stats`	any	Every number for one event (registered, attended, seats left, checked in, certificates…).
`PATCH /events/:id`	write	Partial update (send only changed fields).
`DELETE /events/:id`	write	With registrations → `ARCHIVED` + registration closed (`{ archived: true }`); empty → deleted.
`POST /events/:id/duplicate`	write	Copy + categories as `DRAFT` , title "… (Copy)".
`POST /events/:id/restore`	write	Archived → `COMPLETED` if it has ended, else `DRAFT` .
`GET /categories?eventId=` / `POST { eventId, label }` / `PATCH /:id` / `DELETE /:id`	any / write	Attendee categories. Deleting one leaves its registrations without a category.

Speakers, sponsors, sessions, reorder (`routes/program-routes.ts` , `controllers/program-controller.ts`)

Endpoint	Who	What
`GET /speakers?eventId=&search=&displayPublic=&page=&limit=` , `GET /:id` , `POST` , `PATCH /:id` , `DELETE /:id`	any / write	Speakers of an event. Deleting keeps their sessions (speaker set to null).
`POST /speakers/:id/photo` (multipart `file`)	write	Upload a photo directly for a saved speaker (old file deleted).
`GET /sponsors?eventId=&search=&tier=&displayPublic=…` , CRUD, `POST /sponsors/:id/logo`	any / write	Sponsors with tier.
`GET /sessions?eventId=&search=&displayPublic=…` , CRUD	any / write	Agenda items. A session's speaker must belong to the same event; end after start.
`POST /reorder { type: "speakers" | "sponsors" | "sessions", items: [{ id, displayOrder }] }`	write	All-or-nothing transaction; every id must belong to the org.

`displayPublic` decides what the public event page shows.

Registrations (`routes/registrations-routes.ts` , `controllers/registrations-controller.ts`)

Statuses `REGISTERED | ATTENDED | ABSENT | CANCELLED` . Cancelled registrations don't take a seat (`seatsLeft()`), get no certificate and no emails. `registeredVia` : `WEB` (public page), `ADMIN` , `CSV_IMPORT` , `API` . Every registration is created with a nested ticket (`ticketFor()` : ticket number, QR code, type = category label or "General"). `(eventId, email)` is unique → 409 `DUPLICATE` .

Endpoint	Who	What
`GET /registrations?eventId=&status=&categoryId=&search=&page=&limit=`	any	Paged list with category, ticket and check-in info.
`GET /registrations/export?eventId=&format=csv|xlsx&status=&ids=`	any	CSV (BOM, formula-safe) or Excel (exceljs). Logged.
`POST /registrations`	write	Organizer adds someone (`registeredVia: ADMIN`). Refuses archived or full events.
`POST /registrations/csv-import/parse` (multipart `file` , ≤ 2 MB)	write	Parses with papaparse, returns headers, rows (≤ 5,000) and suggested column mapping.
`POST /registrations/csv-import/confirm { eventId, rows, createMissingCategories?, duplicateStrategy?: first|last|skip, dryRun? }`	write	Validates every row (zod `importRowSchema`), resolves duplicates inside the file, skips existing emails, checks seats, optionally creates categories. `dryRun: true` reports without saving (the wizard's Review step). Returns created/skipped counts and per-row reasons.
`POST /registrations/bulk { action, ids ≤ 1000 }`	write	`mark_attended` , `mark_absent` , `mark_registered` (skip cancelled), `cancel` , `restore` (checks seats per event first), `delete` . Attended → certificate auto-issue in the background.
`POST /registrations/email { ids, subject, message }`	write	Free-text message to the selected people (non-cancelled), sent in the background.
`GET /registrations/:id` , `PATCH /:id`	any / write	Edit fields; changing category also updates the ticket type; restoring from cancelled needs a seat.
`PATCH /registrations/:id/status { status }` (alias `/attendance`)	write	Single status change with the same seat rule; Attended triggers auto-issue.
`DELETE /registrations/:id`	write	Deletes with its ticket.

Uploads (`routes/uploads-routes.ts`)

`POST /uploads/image?kind=speaker|sponsor|logo|signature|payment` (multipart `file`), *write*, expired ok for `payment` , `logo` , `signature` . Branding and payment kinds are admin-only; logos and signatures must be PNG/JPEG. Saves to `uploads/orgs/<orgId>/<speakers|sponsors|branding|payments>/` and returns `{ url }` so forms can preview before saving.

Certificates and verification (`routes/certificates-routes.ts` , `controllers/certificates-controller.ts`)

Types `PARTICIPATION | ACHIEVEMENT | APPRECIATION | SPEAKER | ORGANIZER` . Participation and Achievement need ATTENDED. Unique `(registrationId, type)` → 409 `ALREADY_ISSUED` . Template = request → the event's `certTemplateId` → platform default (`certificates.defaultTemplate`).

Endpoint	Who	What
`GET /certificates/templates`	any	The 60-design catalogue.
`GET /certificates/preview?templateKey=&eventId=&type=`	any	Sample PDF (SAMPLE watermark) with the org's branding.
`GET /certificates?eventId=&status=&type=&emailed=&search=&page=&limit=`	any	List.
`GET /certificates/stats?eventId=`	any	Issued, revoked, emailed, downloads, waiting (attended without one).
`GET /certificates/candidates?eventId=&type=&search=`	write	People who could get this type (for "Issue one").
`POST /certificates { registrationId, type?, templateKey?, sendEmail? }`	write	Issue one (`services/certificates.issueCertificate`).

Endpoint	Who	What
`POST /certificates/bulk { eventId, type?, templateKey?, sendEmail? }`	write	Every ATTENDED registration without this type (max 1,000). Emails in the background, then `notifyCertificatesSent`.
`POST /certificates/email { ids }`	write	(Re)send, revoked skipped, background.
`GET /certificates/:id/pdf?inline=1`	any	Stored PDF; counts a download unless `inline=1`.
`POST /certificates/:id/email`, `/:id/revoke { reason }`, `/:id/restore`	write	Revoke keeps the record with reason; restore clears it.
`GET /verify/:code`	public, 30/min	`valid`, `status` (`ISSUED`, `REVOKED` + reason, or `TAMPERED` when the stored facts no longer match their SHA-256), recipient, event, issuer, date. No email/phone.
`GET /verify/:code/pdf`	public, 30/min	PDF of a valid, untampered certificate (counts a download).

Tickets and check-in (`routes/tickets-routes.ts`, `controllers/tickets-controller.ts`)

Endpoint	Who	What
`GET /tickets?eventId=&search=&used=yes|no&page=&limit=`, `GET /tickets/stats?eventId=`	any	List and numbers.
`GET /tickets/zip?eventId=` or `?ids=`	any	One PDF per ticket in a ZIP (archiver, streamed).
`POST /tickets/email { ids }` or `{ eventId, onlyUnsent? }`	write	Background emailing; sets `emailedAt`.
`GET /tickets/:id/pdf?inline=1`, `POST /tickets/:id/email`	any / write	One ticket.
`POST /tickets/check-in { code, eventId? }`	write	`code` = QR text `EVXT:<qrCode>` or `TKT-…` (`services/tickets.parseScanned`). Refuses wrong event, cancelled registration, archived event. **First scan wins:** `updateMany where isUsed=false` claims the ticket atomically; a second scan gets `ALREADY_USED` with who/when. Sets the registration ATTENDED and triggers auto-issue.
`POST /tickets/:id/undo-check-in`	write	Ticket unused again; ATTENDED → REGISTERED.

ID cards (`routes/idcards-routes.ts`, `controllers/idcards-controller.ts`)

All *any* (printing only reads data). Card options come from the query: `template` (classic, sidebar, minimal, bold), `fields` (category, department, rollNo, email, qr), `logo=0|1`, `layout=sheet|single`; invalid values fall back to defaults.

Endpoint	What
`GET /idcards/options`	Templates and fields for the design panel.
`GET /idcards?eventId=&search=&categoryId=&printed=yes|no&page=&limit=`	Non-cancelled attendees with their last print.
`GET /idcards/stats?eventId=`	Attendees, printed, not printed.
`GET /idcards/preview?…`	Sample people, inline PDF.
`GET /idcards/pdf?eventId=&ids=&…&inline=1`	Real cards; without `inline=1` records an `IdCard` row per person (printed).

Dashboard, analytics, activity, billing, support (`routes/dashboard-routes.ts`)

Endpoint	Who	Controller	What
`GET /dashboard/overview`	any, expired ok	`dashboard-controller`	Cards, 6-month registrations, upcoming events, recent activity.
`GET /analytics`	any	`dashboard-controller` (`analyticsData`)	Everything on the Analytics page (12 months).
`GET /analytics/report`	any	same + `pdf/report.ts`	The same numbers as a 2-page PDF.
`GET /activity?userId=&action=&from=&to=`	admin	`activity-controller`	`action` matches by prefix (`event` → `event.*`).
`GET /activity/export`	admin		CSV, up to 10,000 rows.
`GET /billing`	admin, expired ok	`billing-controller`	Current plan, status, end date, visible plans, payment accounts (setting).
`GET /billing/payments` , `POST /billing/payments` `{ planId, method, referenceNo, proofUrl?, notes? }` , `DELETE /billing/payments/:id`	admin, expired ok		Amount always comes from the plan. One pending payment at a time. Emails the platform contact address. Only pending payments can be cancelled.
`POST /support` `{ subject, message }`	any, expired ok	`support-controller`	Emails the platform contact address with the sender's details.
`GET /contact/info`	public		Contact email, WhatsApp, hours, social links.
`POST /contact`	public, 5/h		Website contact form → platform contact email.

Notifications (`routes/notifications-routes.ts`)

Expired ok. `GET /notifications?limit=` (latest + unread count), `POST /notifications/:id/read` , `POST /notifications/read-all` . Rows are per user; they are created by announcements and `services/orgNotifications.ts` .

Public pages (`routes/public-routes.ts` , `controllers/public-controller.ts`)

No login. Only `PUBLISHED` , `ONGOING` , `COMPLETED` events exist here; suspended organizations are hidden. Uses the white-label name when set. Reads are limited to 120/min per IP.

Endpoint	What
`GET /public/orgs/:slug`	Org branding + upcoming and past events.
`GET /public/events/:id`	Event, availability (`OPEN` , `FULL` , `CLOSED` , `ENDED` , `UNAVAILABLE` for paused orgs), categories, public speakers/sponsors/sessions. Never the meeting link.
`POST /public/events/:id/register` `{ name, email, phone?, department?, rollNo?, categoryId?, website }`	60/hour per IP. `website` is a **honeypot**: if filled, it pretends success and stores nothing. Creates the registration (`WEB`) with its ticket, returns ref/ticket/QR for the on-screen ticket, then in the background emails the confirmation with the ticket PDF and notifies admins (`notifyNewRegistration`). Duplicate email → 409 `ALREADY_REGISTERED` .

API tokens and the REST API (`routes/api-routes.ts`)

Token management (`controllers/api-tokens-controller.ts`): `orgScoped` + admin + `requireApiPlan` (plan `features.restApi`). `GET /api-tokens` , `POST /api-tokens` `{ name, expiresInDays: 30|90|365|null }` → the full `evx_live_…` token **once** (`evx_live_` + 40 hex characters; stored as SHA-256 + a short prefix for display; max 20 active), `POST /api-tokens/:id/revoke` .

REST API (`controllers/ext-api-controller.ts`), `requireApiToken` + 120 requests/min per token. Response shapes are fixed by small mapper functions so internal schema changes don't break integrations; **keep them backwards compatible** (documented publicly at `/developers` , `src/pages/Developers.tsx`). Writes are logged as "via API (token name)".

Endpoint	What
`GET /ext/events?status=upcoming|active|completed|draft|archived&page=&limit=`	Events.
`GET /ext/events/:id`	One event.
`GET /ext/events/:id/registrations?status=&search=&page=&limit=`	Registrations.
`POST /ext/events/:id/registrations { name, email, phone?, department?, rollNo?, category?, sendTicket? }`	Register someone (`API`), category by label, optional ticket email.
`GET /ext/registrations/:id` , `PATCH /ext/registrations/:id { status }`	Read / change status (lowercase names). Attended → auto-issue.
`POST /ext/check-in { code, eventId? }`	Same rules as the dashboard scanner.
`GET /ext/events/:id/certificates` , `GET /ext/certificates/:code`	Certificates of your organization.

Superadmin (`routes/superadmin-routes.ts` , `controllers/superadmin/*`)

All behind `superAdminOnly` . Uses the base `prisma` client (cross-tenant). Platform actions are logged with `organizationId = null` ; actions about a tenant use `entityType: "Organization", entityId` .

`dashboard-controller.ts`

`GET /superadmin/dashboard` : tenant counts by status, revenue (confirmed payments, by confirmation date: this month, year, all time), platform totals, sign-ups per month, "needs attention" items, recent platform activity.

`tenants-controller.ts`

Endpoint	What
`GET /tenants?search=&status=&planId=&from=&to=&sort=&page=&limit=` , `/tenants/stats` , `/tenants/export`	List (search also matches members' emails), counts per status, CSV.
`POST /tenants`	Creates org + first admin with a generated temporary password (returned once, no email).
`GET /tenants/:id`	Drill-down: usage, subscription, seats, team, payments, events, recent activity.
`PATCH /tenants/:id { name?, email?, phone? }`	Edit contact details.
`POST /tenants/:id/plan { planId }`	Change plan (nobody removed).
`POST /tenants/:id/extend { days }` or `{ until }`	Days are added to the later of now and the current end; status becomes the natural one (suspended stays suspended).
`POST /tenants/:id/suspend { reason }` , `/unsuspend`	Suspend with a visible reason; unsuspend → natural status.
`POST /tenants/:id/impersonate`	1-hour token as the org's oldest active admin (`imp` = superadmin id).
`DELETE /tenants/:id { confirm: "<exact name>" }`	Deletes the org (cascade) and its files.

`payments-controller.ts`

`GET /payments` (filters: status, method, plan, org, search, dates), `/payments/stats` , `/payments/export` , `GET /revenue` (12-month chart, by plan, by method, outstanding list). `POST /payments/:id/confirm { note?, notify? }` : only pending; the period starts at the org's current end if it is `active` and still running, otherwise now; +1 year; org gets the payment's plan and `active` (suspended stays suspended); optional renewal email to admins. `POST /payments/:id/reject { note (required), notify? }` : stores the reason in `reviewNote` , optional notice email.

`plans-controller.ts`

`PLAN_FEATURES` lists the on/off features. `GET /superadmin/plans` (with tenant counts), `POST` , `PATCH /:id` , `DELETE /:id` (only unused plans); public `GET /plans` (visible, ordered).

`users-controller.ts`

`GET /users?search=&role=&orgId=&status=`, `/users/stats`, `/users/export`, `GET /users/:id` (profile, recent activity, recent logins). `POST /users/:id/disable` (also bumps `tokenVersion`), `/enable`, `/logout`, `/reset-password { mode: "link" | "temporary" }`, `/reset-2fa`. None of these work on yourself or another superadmin.

`certificates-controller.ts` (platform)

`GET /certificates?search=&status=&type=&orgId=&eventId=`, `/stats`, `/events?orgId=` (for the bulk picker), `/templates`, `/preview?templateKey=`, `GET /:id` (with integrity check: re-hash of the frozen facts), `/:id/pdf` (not counted), `POST /:id/revoke { reason }`, `/:id/restore`, `POST /certificates/bulk-revoke { eventId | orgId, reason, dryRun? }` (dry run returns the count first).

`analytics-controller.ts`

`GET /analytics` (platform numbers), `/analytics/report` (PDF via ReportKit), `/analytics/export?dataset=monthly|plans|organizations|events` (CSV).

`settings-controller.ts`

Settings are flat keys validated by a per-key zod map (unknown keys rejected): `platform.name`, `platform.logoUrl`, `platform.faviconUrl`, `trial.days`, `uploads.maxMb`, `certificates.defaultTemplate`, `contact.email|whatsapp|hours`, `payments.accounts`, `banner`, `seo`, `maintenance.enabled|message`, `website.*` (stats switch, testimonials, logos, social). `GET /superadmin/settings`, `PATCH /superadmin/settings { key: value, … }` (clears the cache), `POST /superadmin/uploads?kind=logo|favicon`. Public: `GET /platform` (what every page needs: name, logo, favicon, banner, maintenance, SEO, trial days, upload limit, contact, social) and `GET /platform/website` (home-page stats if enabled, testimonials, logos).

`emails-controller.ts`

`GET /emails` (each template: default or edited, variables), `PUT /emails/:type { subject, bodyHtml }`, `DELETE /emails/:type` (reset), `POST /emails/:type/preview` and `/test` (render unsaved edits with sample values; test goes to the logged-in superadmin). SMTP: `GET /smtp` (never returns the password), `PUT /smtp { host, port, user, pass?, from? }` (password encrypted; empty keeps the old one), `DELETE /smtp`, `POST /smtp/verify`.

`security-controller.ts`

`GET /security/overview` (24-hour logins/failures, suspicious IPs: 10+ failures or 3+ different emails, currently locked accounts, recent sessions), `/security/activity` (+ `/export`, up to 20,000 rows), `/security/logins`.

`maintenance-controller.ts`

`GET /maintenance` (version from `package.json` + git commit, Node version, uptime, maintenance flags, health checks: DB latency, disk/folder sizes, mail status, job status), `POST /maintenance/jobs/:name/run`, `/cache/clear`, `GET|DELETE /maintenance/errors`, `GET|POST /maintenance/backups` (runs `mysqldump` with the password in the environment, never on the command line; `MYSQLDUMP_PATH` or common install paths), `GET /maintenance/backups/:file` (name validated).

`announcements-controller.ts`

`GET /announcements?status=`, `/stats`, `/audience?planId=` (reach), `POST { title, body, targetPlanId, sendEmail, sendInApp, mode: now|schedule|draft, scheduledAt? }`, `PATCH /:id` (drafts/scheduled), `POST /:id/send`, `/:id/unschedule`, `DELETE /:id` (not sent ones). Delivery is in `services/announcements.ts`.

Activity action codes

`logActivity` codes in use (the frontend's `src/lib/activityText.ts` must have a sentence for each):

`auth.register`, `auth.login`, `auth.logout.all`, `auth.email.verify`, `auth.password.change`, `auth.password.reset`, `auth.profile.update`, `auth.2fa.*` · `org.settings.update`, `org.data.delete`, `org.close` · `team.invite`, `team.invite.accept|cancel|resend`, `team.role.update`, `team.remove` · `event.create|update|delete|archive|restore|duplicate`, `event.attendance.summary` · `speaker.*`, `sponsor.*`, `session.*` · `registration.create|delete|status|self|email|export|csv_import`, `registration.bulk.<action>` · `ticket.checkin`, `ticket.checkin.undo`, `ticket.email`, `ticket.email.bulk`, `ticket.zip` · `certificate.issue|bulk|email|email.bulk|revoke|restore` · `idcard.generate` · `analytics.report` · `upload.image` · `billing.payment.submit|cancel`, `billing.expiry.reminder` · `support.request` ·

`api.token.create|revoke` · superadmin: `tenant.*`, `payment.confirm|reject|export`, `plan.*`, `user.*`, `platform.certificate.*`, `platform.analytics.*`, `settings.update`, `email.template.*`, `email.smtp.*`, `security.export`, `maintenance.*`, `announcement.*`.

5. Services, PDFs and background jobs

Paths are relative to `backend/src/`.

Services (`services/`)

Business logic used by more than one controller or job. Services take the scoped client (`ScopedPrisma`) as a parameter when they work inside one organization, so they stay tenant-safe.

`certificates.ts`

Export	Purpose
`normaliseVerifyCode(input)`	Accepts what people type (lower case, spaces, no dashes) → `EVX-XXXX-XXXX`. Used by verify endpoints and link previews.
`verifyUrl(code)`	`${FRONTEND_URL}/verify/<code>` (printed in the QR and emails).
`certificateHash(facts)`	SHA-256 of a canonical JSON of the facts frozen at issue time: code, recipient name and lower-cased email, type, category, event id, org id, `issuedAt`. If anyone edits those columns in the DB later, verification reports `TAMPERED`.
`certificateData(org, event, cert)`	Builds the renderer input (branding, signatory, event facts, verify URL).
`chooseTemplate(explicit, event)`	Template key: explicit → event's `certTemplateId` → `certificates.defaultTemplate` setting → `classic-royal`.
`issueCertificate(db, org, event, registration, { type, templateKey, issuedAt? })`	Creates the row (verify code, hash, private PDF key), renders the PDF and stores it in `private-files`. If rendering fails the row is deleted (never a certificate without a PDF). Retries up to 3 times on a verify-code collision; a duplicate `(registration, type)` throws a unique error for the caller. `issuedAt` is truncated to whole seconds so the hash survives the DB round trip.
`certificatePdf(org, event, cert)`	The stored PDF, or re-renders and re-stores it if the file is missing (e.g. after moving servers without `private-files/`).
`certificateFileName(cert)`	`Name-EVX-XXXX-XXXX.pdf`.
`emailCertificate(db, org, event, cert)`	Sends the Certificate template with the PDF, sets `emailedAt`.
`EVENT_FOR_CERT`	The Prisma `select` every caller uses for the event.
`autoIssueForAttended(db, org, eventId, registrationIds)`	The attendance hook. If the event has `autoIssueCert`, gives each ATTENDED registration without one a Participation certificate and emails it. Called (in the background) by status changes, bulk actions, check-in and the REST API.

`tickets.ts`

Export	Purpose
`TICKET_INCLUDE`	Prisma include for a ticket with its registration and event.
`ticketPdf(org, ticket)`	Renders on demand (tickets aren't stored, so time/venue changes are always current).
`ticketFileName(ticket)`	`Name-TKT-XXXXXXXXXX.pdf`.
`emailTicket(db, org, ticket)`	Ticket template + PDF, sets `emailedAt`.
`parseScanned(input)`	Scanner/typed input → `{ qrCode }` (from `EVXT:<32 hex>` or bare 32 hex) or `{ ticketNo }` (from `TKT-XXXXXXXXXX`, with or without the dash) or `null`.

`orgNotifications.ts`

Alerts to an organization's active admins, as a bell notification **and** an email (`sendNotice`), controlled by `organizations.notificationPrefs` (each defaults to on; suspended orgs get nothing).

Export	Pref	Triggered by
`notifyNewRegistration(orgId, { name, email, eventTitle, eventId })`	`newRegistration`	public self-registration

Export	Pref	Triggered by
`notifyCertificatesSent(orgId, { sent, failed, eventTitle, eventId })`	`certificate`	end of a background certificate email batch
`notifyAttendanceSummary(orgId, { eventId, title, registered, attended, absent, notMarked })`	`attendance`	`attendance-summaries` job

Each notification has a `link` to the relevant dashboard page; the bell opens it.

`announcements.ts`

Export	Purpose
`audienceWhere(targetPlanId)`	Organizations reached: trial, active and expired (not suspended), optionally one plan.
`deliverAnnouncement(id)`	**Claims** the announcement first (`updateMany` where status is DRAFT/SCHEDULED → SENT), so a scheduled run and a "Send now" click can never both send it. Then creates a notification row per active user of each org (in-app) and, if chosen, emails each org's admins in the background (plain text → escaped paragraphs inside the Announcement template). Stores `orgCount` and `emailCount` .
`sendDueAnnouncements()`	Delivers every SCHEDULED announcement whose time has come (used by the job).

PDF engines (`pdf/`)

All PDFs are drawn with **pdfkit** (no HTML-to-PDF, no headless browser), so they render fast and identically on any server.

`assets.ts`

- `FONTS` and `registerFonts(doc)` : Playfair Display (serif, regular/bold), Great Vibes (script, recipient names), Inter (sans, regular/semibold) loaded from the `@fontsource/*` packages' WOFF files, plus built-in Courier. That's why knip is told to ignore `@fontsource/great-vibes` and `@fontsource/playfair-display` (they're used by file path, not imported).
- `loadImage(url)` : reads `/uploads/...` from disk or fetches an `http(s)` URL (5 s timeout, 5 MB max); returns `null` for anything that isn't PNG/JPEG (pdfkit can't embed WEBP), so a bad logo never breaks a PDF.
- `toBuffer(doc)` : finishes a document into a `Buffer` .

`certificate.ts`

- `LAYOUTS` (10): classic, modern, elegant, minimal, ribbon, bold, academic, geometric, wave, framed.
- `VARIANTS` (5): royal, navy, emerald, crimson, charcoal (primary + gold-ish accent), plus `brand` (the organization's primary/accent colours) → 60 template keys `<layout>-<variant>` . `DEFAULT_TEMPLATE = "classic-royal"` .
- `listTemplates()` , `isTemplateKey(key)` , `CertificateData` , `CertType` .
- `renderCertificate(templateKey, data)` : A4 landscape. Each layout draws its decoration (borders, bands, ribbons, waves…), then a shared text stack (org name, "Certificate of ", recipient in script, event, date/place) measured and centred to fit, signatory block (signature image, name, title), QR to the verify URL with the verify code, and the issue date. `data.sample = true` adds the diagonal SAMPLE watermark for previews. Wording per type is in `TYPE_TEXT` (Speaker and Organizer certificates are titled "Certificate of Appreciation" with their own line, e.g. "is thanked for speaking at").

`ticket.ts`

- `TICKET_QR_PREFIX = "EVXT:"` and `ticketQrValue(qrCode)` : the QR content. The prefix lets the scanner ignore unrelated QR codes.
- `renderTicket(data)` : A6 portrait. Header in the org colour with logo and name, event title, date/time (Asia/Karachi), where (online events: the meeting link; others: the venue), attendee, category, QR (`qrcode` package, error correction M), ticket and registration numbers.

`idcard.ts`

- `IDCARD_TEMPLATES` (classic, sidebar, minimal, bold), `IDCARD_FIELDS` (category, department, rollNo, email, qr), defaults, `IdCardOptions` (`template` , `fields` , `logo` , `layout: sheet | single`).
- `renderIdCards(people, event, options, { sample })` : CR80 cards (54 × 85.6 mm). `single` = one card-sized page per person; `sheet` = 3 × 3 cards per A4 page with crop marks. Names shrink (down to 9 pt, then wrap to 2 lines); detail lines tighten and the least important are dropped if space runs out; text colour adapts to the background (`readableOn`). The QR is the ticket's, so badges work with the check-in scanner. Initials in a circle replace a photo.
- `SAMPLE_PEOPLE` for previews.

`report.ts` (ReportKit)

A small toolkit for A4 reports, reused by the organization analytics report and the superadmin platform report:

Method	Draws
`newReport(title, author)`	a new A4 document with fonts and page buffering (for "Page X of Y")
`new ReportKit(doc, brandColour)`	the cursor-based kit (`y` moves down; `ensure(height)` adds a page when needed)
`header({ title, subtitle, orgName, logo, generatedBy })`	report header with logo
`section(title, description?)`	section heading
`kpis(items)`	a row of KPI tiles
`barChart(items, opts)`	vertical bars (monthly series)
`hBars(items, opts)`	horizontal bars (breakdowns, attendance rates)
`table(columns, rows)`	tables that continue on the next page with the header repeated
`columns(left, right)`	two blocks side by side
`note(text)` , `footer(text)`	small text, page footers

`renderAnalyticsReport(data, { orgName, logoUrl, primaryColor, generatedBy })` builds the 2-page organization report; `controllers/superadmin/analytics-controller.ts` builds the platform one with the same kit. `monthLabel("2026-03") → "Mar"` .

Background jobs (`jobs/`)

`scheduler.ts`

A minimal in-process scheduler:

- `registerJob({ name, label, description, intervalMs, run })` : `run` returns a short summary string.
- `startScheduler()` : called once after `listen` . First runs are staggered (5 s, 7 s, 9 s… after start), then each job runs on its own `setInterval` .
- `runJob(name)` : runs now unless it's already running; records `lastRunAt` , duration, result or error (errors also go to `error_logs`). Used by the interval and by **Maintenance → Run now**.
- `jobStatuses()` : what the Maintenance page shows.

State lives in memory: after a restart the "last run" information starts empty. One API process = one runner; running several processes would run every job several times (see Operations).

`index.ts` (the jobs)

Name	Every	What it does	Run-once guard
`expire-orgs`	10 min	`expireOverdueOrgs()` : trial/active orgs past their end date → expired	idempotent
`expiry-reminders`	6 h	Orgs (trial/active) ending within 3 days: Plan expiring soon email to admins + org email	an `activity_logs` row `billing.expiry.reminder` with `entityId = <end date ISO>` , so each end date is reminded once (a renewal gives a new end date and a new reminder)
`announcements`	1 min	`sendDueAnnouncements()`	the claim in `deliverAnnouncement`
`attendance-summaries`	30 min	Events (public statuses) that ended in the last 2 days: counts by status → `notifyAttendanceSummary`	an `activity_logs` row `event.attendance.summary` per event, written even when the org's pref is off (switching it on later doesn't flood old events)

Adding a job: `registerJob({...})` in `jobs/index.ts` ; make it idempotent (use a marker row or an atomic claim), return a readable summary. It appears on the Maintenance page automatically.

6. Database

MySQL 8 through Prisma 7. Schema: `backend/prisma/schema.prisma`. Migrations: `backend/prisma/migrations/`. Connection: `backend/src/prisma/client.ts` (MariaDB driver adapter, see chapter 3).

Conventions

- **Table names are lowercase snake_case** via `@@map("…")` (MySQL on Linux is case-sensitive for table names). Column names are the Prisma field names (camelCase).
- **Ids** are `cuid()` strings.
- **Tenant column:** every organization-owned table has `organizationId`, indexed, even child tables of `Event`, so tenant filtering never needs a join. These models are listed in `ORG_SCOPED_MODELS` (`src/prisma/scopedClient.ts`); the scoped client adds the filter automatically. A new org-owned model must be added there.
- **Deletes cascade** from `Organization` → users, events, payments, logs, notifications, tokens; from `Event` → categories, registrations, speakers, sponsors, sessions, certificates, tickets, ID cards. Optional links use `SetNull` (a deleted category leaves registrations without one; a deleted speaker leaves sessions without one).
- **Money** is `Decimal(10,2)` in PKR (`currency` column defaults to `"PKR"`). Decimals are sent to the browser as strings.
- **Secrets** are never stored in clear: tokens as SHA-256 hex (`token`, `tokenHash`), 2FA secret encrypted, recovery codes hashed.
- Enum values: auth enums are lowercase (`Role`, `OrgStatus`, historical), everything else UPPER_CASE.

Entity overview

```
Plan 1──* Organization 1──* User ──* PasswordResetToken / EmailVerificationToken
                │            └──* Invitation (invitedBy)
                ├──* Event 1──* EventCategory ──* Registration
                │      ├──* Registration 1──1 Ticket
                │      │         ├──* Certificate   (unique per registration + type)
                │      │         └──* IdCard        (one row per print)
                │      ├──* Speaker 1──* Session
                │      ├──* Sponsor
                │      └──* Session
                ├──* Payment *──1 Plan
                ├──* ActivityLog, Notification, ApiToken, ContactMessage
Platform: SystemSetting, EmailTemplate, Announcement ──* Notification, LoginAttempt, ErrorLog
```

Models

Plans and organizations

Model (table)	Key fields	Notes
`Plan` (`plans`)	`name` unique, `price` (PKR/year), `maxAdmins` / `maxManagers` (null = unlimited), `features` JSON, `isVisible`, `sortOrder`, `description`	Feature keys: `unlimitedEvents`, `unlimitedAttendees`, `qrTicketing`, `bulkCertificates`, `analytics`, `emailAutomation`, `auditLogs`, `restApi`, `customDomain`, `whiteLabel`, `prioritySupport` (+ informational counts). Code checks `features.restApi` for the API.
`Organization` (`organizations`)	`slug` unique (public URL `/o/<slug>`), `status` (trial/active/expired/suspended), `subscriptionEndsAt` (trial and paid end date), `planId`, branding (`logoUrl`, `primaryColor`, `accentColor`, `signatureUrl`, `signatoryName`, `signatoryTitle`), `whiteLabelName`, `customDomain` unique, `notificationPrefs` JSON, `suspendedAt` / `suspendedReason`	The tenant. See the lifecycle in chapter 1.

Auth

Model	Key fields	Notes
`User` (`users`)	`email` unique (one account = one organization), `password` (bcrypt), `role`, `organizationId` (null for superadmin), `emailVerified`, `isActive`, `tokenVersion`, `twoFactorSecret` (encrypted), `twoFactorEnabledAt`, `twoFactorRecoveryCodes` (JSON array of hashes), `lastLoginAt`	`tokenVersion` is in every JWT; incrementing it logs the user out everywhere.
`PasswordResetToken`, `EmailVerificationToken`	`token` unique (hash), `expiresAt`	15 minutes / 24 hours.
`Invitation` (`invitations`)	`(email, organizationId)` unique, `role`, `token` (hash), `expiresAt` (7 days), `acceptedAt`, `invitedById`	Pending = not accepted and not expired. Counts toward seat limits.
`LoginAttempt` (`login_attempts`)	`email`, `userId?`, `success`, IP, user agent	Lockout, security page, suspicious IPs. Indexed by email+time and IP+time.

Events

Model	Key fields	Notes
`Event` (`events`)	`title`, `organizer`, `topic`, `description`, `mode`, `startDateTime` / `endDateTime`, `location`, `meetingLink`, `maxAttendees` (null = no limit), `ticketPrice`, `registrationOpen`, `certTemplateId` (certificate template key; the name is historical), `autoIssueCert`, `status`	Index on `(organizationId, status)`.
`EventCategory` (`event_categories`)	`(eventId, label)` unique	Attendee groups per event.
`Registration` (`registrations`)	`(eventId, email)` unique, `refNo` unique (`REG-…`), name, email, phone, department, rollNo, `categoryId?`, `registeredVia` (WEB/ADMIN/CSV_IMPORT/API), `status`	Index `(eventId, status)`. Always has one `Ticket`.
`Speaker` (`speakers`)	firstName, lastName, title, company, sessionTopic, bio, photo, linkedin, `displayPublic`, `displayOrder`	
`Sponsor` (`sponsors`)	name, website, logo, `tier` (PLATINUM/GOLD/SILVER/BRONZE), `displayPublic`, `displayOrder`	
`Session` (`sessions`)	title, `startTime` / `endTime`, location, `speakerId?`, `displayPublic`, `displayOrder`	Agenda item.

Credentials

Model	Key fields	Notes
`Certificate` (`certificates`)	`(registrationId, type)` unique, `verifyCode` unique (`EVX-XXXX-XXXX`), snapshot fields (`recipientName`, `recipientEmail`, `category`), `templateKey`, `sha256Hash` (CHAR 64), `pdfUrl` (private file key), `status` (ISSUED/REVOKED), `revokedAt` / `revokeReason`, `downloadCount`, `emailedAt`, `issuedAt`	The snapshot + hash make certificates independent of later edits to the registration.
`Ticket` (`tickets`)	`registrationId` unique, `ticketNo` unique (`TKT-…`), `qrCode` unique (32 hex), `type` (category label), `isUsed`, `usedAt`, `usedById`, `emailedAt`	Check-in claims `isUsed` atomically.
`IdCard` (`id_cards`)	`registrationId`, `templateKey`, `fields` JSON, `createdAt`	One row per real download (print history); `pdfUrl` is unused (cards are rendered on demand).

Billing

Model	Key fields	Notes
`Payment` (`payments`)	`planId`, `amount` (copied from the plan), `method` (JAZZCASH/EASYPAISA/BANK_TRANSFER/OTHER), `status` (PENDING/CONFIRMED/REJECTED), `referenceNo`, `proofUrl`, `notes`, `periodStart` / `periodEnd` (set on confirm), `confirmedById` (plain column), `confirmedAt`, `reviewNote`	Revenue = confirmed payments by `confirmedAt`.

Activity, notifications, platform

Model	Key fields	Notes
`ActivityLog` (`activity_logs`)	`organizationId?` (null = platform), `userId?`, `action` (`area.verb`), `entityType` / `entityId`, `metadata` JSON (incl. `impersonatedBy`, `via` for API), IP, user agent	Also used as "done" markers by jobs (`billing.expiry.reminder`, `event.attendance.summary`).
`Notification` (`notifications`)	`organizationId`, `userId`, `announcementId?`, `title`, `body`, `link`, `readAt`	One row per recipient (the API always filters by the current user).
`Announcement` (`announcements`)	`title`, `body` (plain text), `targetPlanId?`, `sendEmail`, `sendInApp`, `status` (DRAFT/SCHEDULED/SENT), `scheduledAt`, `sentAt`, `orgCount`, `emailCount`, `createdById`	
`ApiToken` (`api_tokens`)	`tokenHash` unique (CHAR 64), `prefix` (display), `name`, `expiresAt?`, `revokedAt?`, `lastUsedAt`, `createdById`	
`ContactMessage` (`contact_messages`)	`source` (PUBLIC/DASHBOARD), name, email, orgName, message, `status` (NEW/READ/RESOLVED), `organizationId?`	Every contact-form and Help & support message is stored here **and** emailed to the `contact.email` setting. There is no page that lists them yet; read them in Prisma Studio.
`EmailTemplate` (`email_templates`)	`type` unique, `subject`, `bodyHtml`, `updatedById`	A row exists only for templates the superadmin edited; otherwise the built-in default is used.
`SystemSetting` (`system_settings`)	`key` (PK), `value` JSON	Flat keys (`platform.name`, `trial.days`, `smtp`, `banner`, `website.testimonials` …), validated in `controllers/superadmin/settings-controller.ts`, cached 30 s.
`ErrorLog` (`error_logs`)	message, stack, method, path, status, userId	5xx errors and failed jobs.

Migrations

Folder	What it did
`20261004113153_init`	The whole unified schema (all tables above except the later columns).
`20261011015410_superadmin_panel`	`organizations.suspendedAt/suspendedReason`, `payments.reviewNote`, `plans.description`, `announcements.orgCount/emailCount`, new table `error_logs`. Additive only.
`20261011121810_two_factor_login`	`users.twoFactorSecret/twoFactorEnabledAt/twoFactorRecoveryCodes`. Additive only.

Workflow

```
cd backend
# change schema.prisma, then create + apply a migration on your local database:
npx prisma migrate dev --name short_description
# on servers / other machines (applies pending migrations, never resets):
npx prisma migrate deploy
# after pulling schema changes without migrating (regenerates the client types only):
npx prisma generate
# browse and edit data:
npx prisma studio
```

Rules: never edit an applied migration; prefer additive changes (new nullable columns, new tables) so a deploy can't break running code; `prisma migrate reset` wipes the database (ask first, see CLAUDE.md).

Useful queries (Prisma Studio or SQL)

- Who is the superadmin? `users` where `role = 'superAdmin'`.
- Why can't an org log in? `organizations.status` and `suspendedReason`; their `subscriptionEndsAt`.
- Did a reminder go out? `activity_logs` where `action = 'billing.expiry.reminder'` and `organizationId = …`.
- Contact messages: `contact_messages` ordered by `createdAt` desc.

7. Frontend core

The infrastructure every page uses. Paths are relative to `src/`. The `@/` import alias points to `src/`.

Entry: `main.tsx`

1. Imports `index.css`.
2. **Stale-chunk recovery:** after a deploy, an open tab may request a page file that no longer exists; Vite fires `vite:preloadError`. The handler reloads the page once (a `sessionStorage` flag prevents a reload loop; it is cleared 10 seconds after a successful load).
3. Creates a **data router** (`createBrowserRouter`) with one catch-all route whose element is the provider stack `PlatformProvider → AuthProvider → ConfirmProvider → App`. A data router is required by `useBlocker` (unsaved-changes prompts); the actual routes stay declarative in `App.tsx`.

Routes: `App.tsx`

- **65 pages, all** `React.lazy`, so each page is its own JS file. `Suspense` shows `PageLoader` while one loads.
- `Layout`: public pages get `Navbar` + `SiteBanner` + `Footer`. Paths in `BARE_PREFIXES` (`/dashboard`, `/superadmin`, `/e/`, `/o/`, auth pages, `/dev`) render without them (they bring their own layout).
- `ScrollToTop` scrolls up on every path change.
- Route groups:
  - public: `/`, `/about`, `/features`, `/pricing`, `/contact`, `/terms`, `/privacy`, `/verify`, `/verify/:code`, `/developers`, `/o/:slug`, `/e/:id`;
  - guest-only (`GuestRoute`, logged-in users are sent home): `/login`, `/register`, `/forgot-password`;
  - token links: `/reset-password`, `/verify-email`, `/accept-invite`;
  - `/dashboard/*` inside `ProtectedRoute roles={admin, manager, viewer}` + `DashboardLayout`; admin-only children wrapped again (`activity`, `billing`, `api`); create/edit pages wrapped in `writer(page)` = admins and managers;
  - `/superadmin/*` inside `ProtectedRoute roles={superAdmin}` + `SuperadminLayout`;
  - `/dev/components` (component gallery) only in development (`import.meta.env.DEV`);
  - `*` → `NotFound`.

`components/auth/RouteGuards.tsx`

Export	Behaviour
`ProtectedRoute({ roles? })`	While auth is loading: full-page spinner. Guest: redirect to `/login` with `state.from` (login returns there). Wrong role: redirect to `homeFor(user)`.
`GuestRoute`	Logged in → redirect to `homeFor(user)`.
`FullPageLoader`	The spinner screen.

`homeFor(user)` (in `AuthContext`) is `/superadmin` for superadmins, `/dashboard` for everyone else.

Talking to the backend: `lib/api.ts`

The **only** module that calls `fetch`.

- `API_BASE` = `import.meta.env.VITE_API_URL` or `/api/v1` (same origin; Vite proxies it in development, the backend serves it in production). Set `VITE_API_URL` at build time only if the API lives on another origin.
- **`tokenStore`**: the JWT is stored under `evx_token` in `localStorage` ("Remember me") or `sessionStorage`; session storage wins when both exist. Impersonation: `startImpersonation(token)` saves the superadmin's own token under `evx_return_session` (session storage, this tab only) and puts the impersonation token in session storage; `stopImpersonation()` restores it; `isImpersonating()`.
- **Requests** add `Authorization: Bearer …` unless `{ auth: false }`; JSON bodies are serialized; `FormData` is sent as is.
- **Responses:** understands the `{ data, error, meta }` envelope and the plain JSON of `/auth`. Every failure becomes an `ApiError(message, status, code, fieldErrors?)`; network failures become `NETWORK_ERROR` ("Is the backend running?").
- **Events:** a 401 on a request that carried a token notifies `onUnauthorized` listeners (AuthContext logs out, or returns to the superadmin after an impersonation ends); a 503 `MAINTENANCE` notifies `onMaintenance` listeners (PlatformContext refreshes and shows the maintenance screen).
- **Methods:** `api.get`, `api.list` / `api.getList` (returns `{ data, meta }` for paged lists; two names for historical reasons), `post`, `patch`, `put`, `delete`, `upload(path, file, field = "file")`, `blob(path)` (authenticated file as a Blob, e.g. PDF preview), `download(path,`

`fallbackName)` (authenticated download using the server's file name).

- Helpers: `buildQuery({ a, b })` (skips empty values), `errorMessage(err, fallback)`.

Contexts (`context/`)

`AuthContext.tsx`

State: `status` (`loading` | `guest` | `authenticated`), `user`, `organization` (with plan), `impersonator`.

Function	Does
`login(email, password, remember)`	`POST /auth/login`; returns `{ user }` or `{ challengeToken }` when 2FA is needed
`completeTwoFactor(challengeToken, code, remember)`	`POST /auth/login/2fa`
`register(input)`, `acceptInvite(input)`	create the account and start a remembered session
`logout()`	clears the stored token (JWTs are stateless: use "log out everywhere" to invalidate other sessions)
`refresh()`	reloads `GET /auth/me` (after changing org settings, plan, profile…)
`impersonate(token)`, `stopImpersonating()`	switch to / back from a "log in as" session

On start, if a token exists, it calls `/auth/me`; the token is only thrown away if the server answers 401/404 (not when the server is down).

`PlatformContext.tsx`

Loads `GET /platform` (name, logo, favicon, banner, maintenance, SEO, trial days, upload limit, contact, social) on start, every 60 seconds and whenever an API call reports maintenance. Applies the favicon, `document.title` and meta description. `usePlatform()` → `{ platform, refresh }`. Used by the banner, maintenance screen, footer, sign-up page, upload fields.

`components/app/ConfirmDialog.tsx` (`ConfirmProvider` , `useConfirm`)

`const confirm = useConfirm(); if (await confirm({ title, description, confirmLabel, cancelLabel?, tone: "danger" })) …` One dialog for the whole app, promise-based. Used for every destructive action and by `useUnsavedChanges`.

Hooks (`hooks/`)

Hook	Use
`useApi<T>(path | null)`	Loads `path` → `{ data, meta, loading, initialLoading, error, reload }`. `null` skips (e.g. no event selected). A new path aborts the previous request (no stale results while typing). `initialLoading` is true only before the first data, so reloads keep showing old rows instead of flashing a skeleton.
`useUrlState(defaults)`	Filters/search/page in the URL: `{ values, set(changes), reset(keep = ["eventId"]) }`. Defaults are omitted from the URL; any change except `page` resets to page 1; uses `replace` (no history spam).
`useSelectedEvent()`	`[eventId, select]` for the `EventPicker`: `?eventId=` in the URL, remembered in `localStorage` so Registrations → Speakers → Certificates keep the same event. Selecting an event clears other filters.
`useUnsavedChanges(dirty)`	In-app navigation asks "Discard unsaved changes?" (`useBlocker` + `useConfirm`); closing/refreshing the tab shows the browser warning. React Router allows **one** blocker at a time, so a page with several forms (Settings) owns a single call.
`useVisibilityToggle(resource, onChanged)`	Show/hide a speaker, sponsor or session on the public page with an Undo toast.
`useNow(intervalMs = 60000)`	Current time that re-renders periodically (countdowns, "time left").

Helpers (`lib/`)

File	Exports
`status.ts`	**Single source** of every status label and colour: `EVENT_STATUS`, `EVENT_MODE`, `REGISTRATION_STATUS`, `SPONSOR_TIER`, `ORG_STATUS`, `TEAM_ROLE`, `PAYMENT_STATUS`, `CERT_STATUS`, `CERT_TYPE`, `ANNOUNCEMENT_STATUS`, `VISIBILITY`; `STATUS_MAPS`, `TONE_CLASSES`, `statusOptions(kind)` (for filters), `statusLabel(kind, value)`. Used by `StatusBadge`. Change wording here, e.g. PUBLISHED → "Upcoming".

File	Exports
`format.ts`	`formatDate`, `formatDateTime`, `formatTime`, `formatPKR` ("Free" for 0), `formatMoney` (always an amount, "PKR 0"), `formatDuration`, `formatRelative` ("5 min ago"), `formatTimeLeft`, `formatMonth("2026-10")`, `plural(n, word)`. Locale `en-PK`.
`permissions.ts`	`useCan()(permission)`: `write` = admin/manager (and superadmin), `manage` = admin (and superadmin).
`validation.ts`	zod `emailSchema`, `passwordSchema`, `phoneSchema` (Pakistani numbers) and `PASSWORD_HINT`, mirroring the backend rules.
`activityText.ts`	`describeActivity(item)`: activity action code + metadata → sentence ("created the event "Tech Summit""). **Add a case for every new action code.**
`date.ts`	`toDatetimeLocal(iso)` / `fromDatetimeLocal(value)` for `<input type="datetime-local">`.
`csv.ts`	`downloadCsv(fileName, header, rows)` (client-side CSV with BOM).
`contact.ts`	`whatsappLink(number, text?)` → `https://wa.me/…`.
`utils.ts`	`cn(...classes)` (clsx + tailwind-merge), used by every component.

Types (`types/`)

TypeScript shapes of API responses, one file per area: `auth` (Role, AuthUser, Organization, Plan, responses), `event`, `registration`, `speaker`, `sponsor`, `session`, `ticket`, `certificate`, `idcard`, `dashboard` (overview, analytics, activity), `team`, `billing`, `platform` (`/platform` settings), `website` (home-page content), `public` (public org/event pages), `superadmin` (tenants, payments, users, security, maintenance, announcements…). They are written by hand to match the controllers; when an endpoint's response changes, update its type.

Styles: `index.css`, Tailwind

- Google Fonts: **Playfair Display** (headings, `.font-heading`) and **Inter** (body, `.font-body`).
- `@layer base`: shadcn CSS variables, body font and background.
- `@layer components`: `.font-heading`, `.font-body`, `.btn-primary` and similar public-site helpers; `@layer utilities`: named colour helpers from the original homepage (`.text-purple`, `.bg-lavender` …). Then the home page's animations (ticker columns, float, shimmer, 3D text), with `prefers-reduced-motion` respected.
- `tailwind.config.js`: brand purple scale (`primary` = `#7c3aed`), shadcn colour tokens, `tailwindcss-animate`.
- Colour conventions in components use literal hex values from CLAUDE.md (`#7c3aed`, `#0f172a`, `#64748b`, `#e9e4ff`, `#f3f0ff`).

8. Frontend components

Paths are relative to `src/components/`. Three layers, from generic to specific:

1. `ui/`: visual primitives (shadcn/ui on Radix, restyled to the EventoraX look). No app logic.
2. `app/`: app-wide building blocks that know about our conventions (server pagination, URL filters, statuses, roles).
3. `layout/` and feature folders: the dashboard frame and pieces used by one area.

Before writing a new component, look here first. Most pages are composed almost entirely from `app/`. `/dev/components` (development only) shows the shared components live (`pages/dev/ComponentGallery.tsx`).

`ui/` (primitives)

File	What
`button.tsx`	`Button` (+ `buttonVariants`): variants `default` / `primary` (purple), `destructive` / `danger`, `outline`, `secondary`, `ghost`, `link` ..., sizes incl. `icon` / `icon-sm`; `asChild` to render a `Link`.
`form-fields.tsx`	**Use these in forms**: `TextField`, `PasswordField` (show/hide), `SelectField` (options array), `TextareaField`. Each renders label, required star, helper text and the error message under the field, and works with `register()` from react-hook-form.
`input.tsx`, `select.tsx`, `checkbox.tsx`, `switch.tsx`, `tabs.tsx`	Raw inputs (Radix Select, Checkbox, Switch, Tabs).
`dialog.tsx`, `alert-dialog.tsx`, `sheet.tsx`, `popover.tsx`, `dropdown-menu.tsx`, `tooltip.tsx`	Overlays (modal dialog, confirmation dialog, side sheet, popover, menus, tooltips).
`card.tsx`	`Card` (variants) with header/title/description/content/footer/action parts.
`table.tsx`	Table parts used by `ServerTable`.
`skeleton.tsx`, `spinner.tsx`	Loading placeholders.
`sonner.tsx`	The `Toaster` (toasts via `toast.success/error(..., { action: { label: "Undo", onClick } })`).

Unused extra exports of these files are allowed (knip ignores exports in `ui/`). Add new primitives with `npx shadcn add <name>` (configured by `components.json`), then restyle to match.

`app/` (building blocks)

Page structure

Component	Use
`PageHeader({ title, description?, breadcrumbs, actions?, badge? })`	Top of every dashboard page: breadcrumbs, title, one-line description (list pages only), actions on the right.
`EventPicker`	Chooses the event for event-scoped pages (with `useSelectedEvent`). Archived events listed last.
`StatsRow({ items, loading })`, `StatCard`	2–4 stat cards with icon, accent colour and hint; skeletons while loading.
`SegmentedTabs({ value, onChange, options })`	Main filter as tabs, with counts and optional dot (unsaved).
`Panel({ title, description?, action? })`, `PanelLink`	White titled card for overview/analytics pages.
`PageLoader`	Spinner while a lazy page loads.

Lists

Component	Use
`Toolbar({ children, end })`	Row above a list.
`SearchInput({ value, onChange })`	Debounced (300 ms) search box.

Component	Use
`FilterSelect({ value, onChange, options, allLabel })`	Dropdown filter; empty value = "All".
`FilterChips({ filters, onClearAll }) + ActiveFilter`	Active filters as removable chips.
`ViewToggle({ value: "table" | "grid", onChange })`	List / card view switch.
`DateRangeFilter` + `dayStart` / `dayEnd`	From–to dates; converts days to exact ISO start/end in the viewer's time zone.
`ServerTable<T>({ columns, rows, rowKey, loading, selectedIds?, onToggleRow?, onToggleAll?, onRowClick? }) + Column<T>`	Table for server-paginated data: columns with `cell`, `align`, `hideBelow` (responsive), optional checkbox selection for bulk actions.
`CardGrid`	Responsive grid with skeletons (card view).
`PaginationBar({ page, limit, total, onPageChange, onLimitChange })`	"Showing 21–40 of 132", rows per page, previous/next.
`BulkActionBar({ count, onClear, children })`	Appears when rows are selected.
`RowActions({ actions }) + RowAction`	The "⋯" menu: `label`, `icon`, `onClick`, `destructive`, `hidden`, `separatorBefore`.
`StatusBadge({ kind, value })`	Coloured pill from `lib/status.ts`.
`States.tsx`: `EmptyState`, `NoResults`, `ErrorState`	Nothing yet (with call to action), filters matched nothing (Clear), loading failed (Retry).
`Avatar({ name, src })`	Photo or initials.
`ReorderList<T>`	Drag-and-drop ordering (dnd-kit; mouse, touch, keyboard) with Move up/down buttons; used by the reorder pages.

Permissions and dialogs

Component	Use
`RoleGate({ need: "write" | "manage", fallback? })`	Renders children only for allowed roles (hides buttons from viewers).
`ConfirmDialog.tsx`: `ConfirmProvider`, `useConfirm()`	Promise-based confirmation (see chapter 7).

Forms (`app/form/`)

Export	Use
`FormSection({ title, description })`	Titled card with a two-column field grid.
`FullWidth`	A field spanning both columns.
`SwitchField`	On/off setting with label and explanation.
`FormFooter({ submitting, onCancel, submitLabel?, dirty? })`	Cancel / Save row at the end of the form (not sticky).
`ImageUploadField({ label, kind, value, onChange, uploadUrl?, helper?, error? })`	Drag-and-drop or click to upload, preview, remove, or "Use a link instead"; respects the platform upload limit; uploads to `/uploads/image?kind=…` (or a custom `uploadUrl`) and stores the returned URL.
`applyServerErrors(err, setError)`	Puts the API's `fieldErrors` under the matching fields and returns a message for a toast.

Form page recipe (used by every create/edit page):

```
const form = useForm<Values>({ resolver: zodResolver(schema), defaultValues })
const [goTo, setGoTo] = useState<string | null>(null)
useUnsavedChanges(form.formState.isDirty && goTo === null)
useEffect(() => { if (goTo) navigate(goTo) }, [goTo])          // navigate after the guard is off
async function onSubmit(values) {
  try { await api.post(...); toast.success("Saved"); setGoTo("/dashboard/...") }
  catch (err) { toast.error(applyServerErrors(err, form.setError)) }
}
```

`layout/` (dashboard frame)

File	What
`DashboardLayout.tsx`	Frame for `/dashboard/*`: fixed `Sidebar` (collapsible, remembered in `localStorage` `evx_sidebar_collapsed`), phone drawer, `Topbar`, `ImpersonationBanner`, `SiteBanner`, `AccountBanners`, then the page (`<Outlet/>`, padding `p-4 sm:p-6`). Shows `MaintenanceScreen` instead when maintenance is on (except while impersonating).
`nav.ts`	`NAV` (organization menu, groups Events / Credentials / Insights / Organization, items with `need: "write" | "manage"`) and `SUPERADMIN_NAV`. **Add new pages here.**
`Sidebar.tsx`	Renders a nav (`variant="platform"` for superadmin), hides items the role can't use, plan card with time left at the bottom, collapse button.
`Topbar.tsx`	Phone menu button, `NotificationBell` (organizations), account menu (settings, log out).
`NotificationBell.tsx`	Unread count, latest 15, click to read/expand/open the link, Mark all as read; refreshes every minute.
`AccountBanners.tsx`	Trial running / plan ending soon / expired (with Billing link), and "verify your email" with Resend.
`ImpersonationBanner.tsx`	Dark strip "Viewing as … · Back to superadmin" during "log in as".
`SiteBanner.tsx`	The sitewide banner from System settings; dismissing hides that exact message (stored per message).
`MaintenanceScreen.tsx`	"We'll be right back" with the superadmin's message.

`superadmin/SuperadminLayout.tsx` is the same frame with `SUPERADMIN_NAV` and an amber strip while maintenance mode is on.

Feature folders

Folder	Components
`events/`	`EventForm` (create/edit, 5 sections, certificate design picker, categories), `CategoriesEditor` (chips + suggestions; edits existing categories through the API when editing an event), `EventCard` (card view), `CapacityBar` (amber near full, red when full), `PublicLinkCard` (public URL, copy, poster QR PNG, open; explains drafts), `useEventActions` (duplicate / archive / restore / delete with confirmations and toasts, shared by the list and the hub).
`registrations/`	`RegistrationForm` (add/edit attendee), `StatusMenu` (badge that opens a status menu; read-only for viewers), `EmailAttendeesDialog` (subject + message to selected people).
`speakers/`, `sponsors/`, `sessions/`	`SpeakerForm`, `SponsorForm` (tier, logo), `SessionForm` (time range, speaker from the event, room).
`team/`	`InviteDialog` (email + role cards with seats left), `roles.ts` (`ROLE_INFO`, `seatsLeft`).
`settings/`	One component per Settings tab: `AccountTab` (profile, password, `TwoFactorSection`, sessions), `OrganizationTab`, `BrandingTab` (logo, colour pickers synced with hex inputs, preview, white label + domain), `CertificatesTab` (signatory + signature with preview), `NotificationsTab` (switches save immediately, with Undo), `DangerZoneTab` (typed name + password). `shared.tsx`: `useReportDirty` (tabs report unsaved state to the page's single guard), `SaveBar`, `saveOrg(fields)` (PATCH `/org/me` + refresh auth). `TwoFactorSection`: set-up flow (QR, code, recovery codes shown once with copy/download), turn off, new codes.
`billing/`	`PayDialog` (step 1 where to pay, step 2 plan/method/transaction ID/receipt/notes), `paymentInfo.tsx` (`METHOD_LABEL`, `FEATURE_LABELS`, `PaymentAccountsList`, `hasAccounts`).
`certificates/`	`TemplatePickerDialog` (10 layouts × colours with a live PDF preview), `IssueDialogs.tsx` (`IssueOneDialog` with attendee search, `BulkIssueDialog` with the eligible count, `RevokeDialog` with reason), `pdf.ts` (`usePdfUrl(path)` fetches an authenticated PDF into a blob URL for `<iframe>` previews; `openPdf(path)` opens one in a new tab without being blocked as a pop-up).
`tickets/`	`QrCode` (QR drawn in the browser with `qrcode`; `ticketQrValue` must match the backend prefix `EVXT:`), `TicketDialog` (big QR, PDF, email again, check in manually / undo).
`idcards/`	`DesignPanel` (template thumbnails, fields, logo, layout, live preview, full-page preview), `design.ts` (`useIdCardDesign` stored in `localStorage`, `designQuery` → query string for preview/PDF endpoints).
`superadmin/`	`SuperadminLayout`, `ActivityFeed`, `PaymentReviewDialog` (details, receipt, confirm with note + email switch, reject with reason), `TenantDialogs.tsx` (`CreateTenantDialog`, `ChangePlanDialog`, `ExtendDialog`, `SuspendDialog`, `OneTimeSecret` for temporary passwords), `WebsiteContentPanel` (live stats switch, testimonials, logos, social links), `useImpersonate` (confirm → `POST /tenants/:id/impersonate` → `impersonate(token)` → open the dashboard).

Folder	Components
`public/`	`plans.tsx` (`usePublicPlans` , `useTrialDays` , `PlanCards` , `PlanComparison` , `PaymentBadges`), `PublicShell` + `brandColor` (organization-branded frame for `/e/` and `/o/` pages), `LegalPage` (Terms/Privacy layout with table of contents and the contact email), `SocialLinks` .
`home/`	Landing page sections used by `pages/Home.tsx` : `HeroSection` (loads `OceanCanvas` lazily), `TrustedBySection` , `StatsSection` , `HowItWorksSection` , `FeaturesSection` , `PricingSection` , `TestimonialsSection` , `FAQSection` , `CTASection` . Sections fed by System settings → Website hide themselves when empty.
`auth/`	`AuthShell` (centred card layout of login/register/reset pages), `RouteGuards` (chapter 7).
(root)	`Navbar` (public top menu, mobile menu, Login/Get Started or Dashboard when logged in), `Footer` (links, contact info and social links from settings, maintenance status dot, year), `OceanCanvas` (three.js animated waves for the hero; its own lazy chunk, ~500 kB).

9. Frontend pages

One file per route in `src/pages/`. Every page is lazy-loaded from `App.tsx`. For what each page looks like and does from a user's point of view, see the product guide; this chapter covers how each is built and which endpoints it uses.

Patterns used by almost every page (details in chapters 7 and 8):

- **List page:** `PageHeader` → `EventPicker` (event-scoped pages) → `StatsRow` (a `/stats` endpoint) → `SegmentedTabs` / `Toolbar` with state in `useUrlState` → `FilterChips` → `BulkActionBar` → `ServerTable` or `CardGrid` → `PaginationBar`, with `ErrorState` / `NoResults` / `EmptyState`. Data via `useApi(path)` where `path` is built with `buildQuery(values)`, so changing a filter changes the URL, which changes the path, which reloads.
- **Form page:** a thin page (`New…Page`, `Edit…Page`) that loads the record (edit) and renders the feature's form component (`EventForm`, `RegistrationForm`, …), which follows the form recipe in chapter 8.
- Write actions are wrapped in `RoleGate need="write"`; destructive ones use `useConfirm`.

Public website (`pages/*.tsx`)

Route	File	Notes
`/`	`Home.tsx`	Composes `components/home/*`. Loads `GET /platform/website` (live stats, testimonials, logos) and passes them to sections, which hide when empty. Pricing section loads `GET /plans`.
`/features`	`Features.tsx`	Static content (keep it true to the product).
`/pricing`	`Pricing.tsx`	`usePublicPlans` (`GET /plans`), `PlanCards`, `PlanComparison`, `PaymentBadges`, billing FAQ; trial length from `usePlatform`.
`/about`	`About.tsx`	Static; **team names are placeholders** (see handover).
`/contact`	`Contact.tsx`	`GET /contact/info`, `POST /contact` (rate-limited), social links.
`/terms`, `/privacy`	`Terms.tsx`, `Privacy.tsx`	Content arrays rendered by `LegalPage`.
`/verify`, `/verify/:code`	`Verify.tsx`	Code input (normalised), `GET /verify/:code`, VERIFIED / REVOKED / TAMPERED / not found, PDF via `/verify/:code/pdf`.
`/developers`	`Developers.tsx`	REST API documentation (base URL from `window.location.origin`). **Update it when `/ext` changes.**
`*`	`NotFound.tsx`	404 page.

Organization public pages (`pages/public/`)

Route	File	Notes
`/o/:slug`	`OrgPage.tsx`	`GET /public/orgs/:slug` → upcoming and past events inside `PublicShell` (org colours/logo).
`/e/:id`	`EventPage.tsx`	`GET /public/events/:id` (unauthenticated). Hero with facts and availability, schedule, speakers, sponsors by tier; **Share** (`navigator.share` or copy link); **Add to calendar** (builds an `.ics` file in the browser); registration form (react-hook-form + zod, hidden honeypot `website` field) → `POST /public/events/:id/register` → shows the QR ticket on screen (`QrCode`). Handles FULL / CLOSED / ENDED / UNAVAILABLE and "already registered".

Link-preview tags for these two routes and `/verify/:code` are added by the backend (`web/spa.ts`), not here.

Authentication pages

Route	File	Notes
`/login`	`Login.tsx`	Two steps: password (`useAuth().login`), then, if `challengeToken` comes back, a 6-digit code or recovery code (`completeTwoFactor`). Returns to `location.state.from` or `homeFor(user)`. Shows suspension reasons and lockout messages from the API.
`/register`	`Register.tsx`	Sign-up form (`useAuth().register`), trial length from settings, terms checkbox.
`/forgot-password`	`ForgotPassword.tsx`	`POST /auth/forgot-password`, always the same confirmation.
`/reset-password?token=`	`ResetPassword.tsx`	`POST /auth/reset-password`.

Route	File	Notes
`/verify-email?token=`	`VerifyEmail.tsx`	`GET /auth/verify-email`, then refreshes the session if logged in.
`/accept-invite?token=`	`AcceptInvite.tsx`	`GET /auth/invite/:token` (who invited you), then `acceptInvite`.

All use `AuthShell` for the centred layout.

Organization dashboard (`pages/dashboard/`)

Route	File	Endpoints	Notes
`/dashboard`	`OverviewPage.tsx`	`GET /dashboard/overview`	Greeting, stat cards, 6-month area chart (recharts), quick actions (role-aware), upcoming events, recent activity. Works when expired.
`/dashboard/events`	`events/EventsPage.tsx`	`/events`, `/events/stats`	Status tabs with counts, search, mode, date range, table/card view, row actions via `useEventActions`.
`/dashboard/events/new`, `/:id/edit`	`events/NewEventPage.tsx`, `EditEventPage.tsx`	`/events/:id`	`EventForm`.
`/dashboard/events/:id`	`events/EventDetailPage.tsx`	`/events/:id`, `/events/:id/stats`, lazy previews of `/registrations`, `/speakers`, `/sponsors`, `/sessions`	The hub: `PublicLinkCard`, stat cards, tabs in `?tab=` (each tab's data loads only when opened), edit/duplicate/archive. Archived = read-only.
`/dashboard/registrations`	`registrations/RegistrationsPage.tsx`	`/registrations`, `/events/:id/stats`, `/registrations/:id/status`, `/registrations/bulk`, `/registrations/export`, `/registrations/email`, `/categories`	Status tabs, search, category filter, inline `StatusMenu`, selection + bulk bar, export menu (CSV/Excel, all or selected), `EmailAttendeesDialog`.
`/dashboard/registrations/new`, `/:id/edit`	`NewRegistrationPage.tsx`, `EditRegistrationPage.tsx`	`/registrations/:id`	`RegistrationForm`.
`/dashboard/registrations/import`	`registrations/ImportRegistrationsPage.tsx`	`/registrations/csv-import/parse`, `/registrations/csv-import/confirm`	4-step wizard (`upload` → `map` → `review` → `done`). Column aliases guess the mapping (e.g. "ticket type" → category). Review calls confirm with `dryRun: true`; Import calls it for real. Template and skipped-rows CSVs are built in the browser.
`/dashboard/speakers` (+ `new`, `:id/edit`, `reorder`)	`speakers/*`	`/speakers`, `/reorder`	Card/list view, public/hidden filter, `useVisibilityToggle`, `ReorderList`.
`/dashboard/sponsors` (+ `new`, `:id/edit`)	`sponsors/*`	`/sponsors`	Grouped by tier.
`/dashboard/schedule` (+ `new`, `:id/edit`, `reorder`)	`schedule/*`	`/sessions`, `/reorder`	Timeline grouped by day, totals.

Route	File	Endpoints	Notes
`/dashboard/certificates`	`certificates/CertificatesPage.tsx`	`/certificates`, `/certificates/stats`, `/templates`, `/:id/pdf`, `/:id/email`, `/email`, `/:id/revoke`, `/:id/restore`, `PATCH /events/:id`	Design card (`TemplatePickerDialog` saves `certTemplateId` on the event), auto-issue switch (with Undo), issue dialogs, row menu, revoke dialog. PDFs open via `openPdf`.
`/dashboard/tickets`	`tickets/TicketsPage.tsx`	`/tickets`, `/tickets/stats`, `/tickets/zip`, `/tickets/email`, `/tickets/:id/pdf`, `/tickets/check-in`, `/:id/undo-check-in`	`TicketDialog` for one ticket, ZIP download, "email unsent".
`/dashboard/tickets/scan`	`tickets/ScanPage.tsx`	`/tickets/check-in`, `/tickets/stats`, `/:id/undo-check-in`	`qr-scanner` on a `<video>` (camera choice, torch), ignores the same code for 3 s (the camera sees it many times a second), one request at a time, beep (WebAudio) + vibration for success/problem, manual ticket-number box, live counters, recent scans with Undo. Needs `https://` or localhost for the camera.
`/dashboard/idcards`	`idcards/IdCardsPage.tsx`	`/idcards`, `/idcards/stats`, `/idcards/options`, `/idcards/preview`, `/idcards/pdf`	`DesignPanel` (design in localStorage), attendee table with printed filter, download all/selected/one.
`/dashboard/analytics`	`AnalyticsPage.tsx`	`/analytics`, `/analytics/report`	recharts charts; PDF report download.
`/dashboard/activity` (admin)	`ActivityPage.tsx`	`/activity`, `/activity/export`, `/team`	Filters by person (team list), action prefix, date range; sentences from `describeActivity`.
`/dashboard/team`	`TeamPage.tsx`	`/team`, `/team/invite`, `/team/:id/role`, `/team/:id`, `/team/invites/:id(/resend)`	Seat cards, role select with Undo toast, remove with confirm, pending invites, `InviteDialog`. Read-only for non-admins.
`/dashboard/settings`	`SettingsPage.tsx`	via tabs: `/auth/me`, `/auth/change-password`, `/auth/2fa/*`, `/auth/logout-all`, `/org/me`, `/org/me/delete-data`, `/org/me/close`	Tabs from `components/settings/*` (non-admins only see My account). Every tab stays mounted; each reports `onDirtyChange`; the page owns the single `useUnsavedChanges`. Tab in `?tab=`.
`/dashboard/billing` (admin)	`BillingPage.tsx`	`/billing`, `/billing/payments` (GET/POST), `DELETE /billing/payments/:id`	Current plan card, plan cards with Pay/Renew/Switch, `PayDialog`, payment history with receipts and rejection reasons.
`/dashboard/api` (admin)	`ApiPage.tsx`	`/api-tokens`, `/api-tokens/:id/revoke`	Enterprise: base URL + curl example, token list, create dialog with the one-time token. Other plans: upsell to Billing.

Route	File	Endpoints	Notes
`/dashboard/contact`	`ContactPage.tsx`	`/contact/info`, `POST /support`	Message form, WhatsApp/email/hours, FAQ.

Superadmin (`pages/superadmin/`)

Route	File	Endpoints	Notes
`/superadmin`	`SuperadminDashboardPage.tsx`	`/superadmin/dashboard`	KPI cards, revenue, needs-attention links, sign-ups chart, `ActivityFeed`, quick actions.
`/superadmin/tenants`	`TenantsPage.tsx`	`/superadmin/tenants`, `/stats`, `/export`, `/plans`	Status tabs, filters, sort, CSV; row menu uses `TenantDialogs` and `useImpersonate`.
`/superadmin/tenants/:id`	`TenantDetailPage.tsx`	`/superadmin/tenants/:id` (+ actions)	Drill-down with actions bar and danger zone (typed-name delete).
`/superadmin/revenue`	`RevenuePage.tsx`	`/superadmin/revenue`, `/payments`, `/payments/stats`, `/payments/export`, `/plans`	Charts, Transactions / Outstanding views in `?view=`, `PaymentReviewDialog`. `?orgId=` filters to one tenant.
`/superadmin/plans`	`PlansPage.tsx`	`/superadmin/plans` (CRUD)	Plan cards, editor dialog with feature switches, hide/show with Undo, feature matrix.
`/superadmin/users`	`UsersPage.tsx`	`/superadmin/users` (+ actions), `/stats`, `/export`	Side panel in `?user=`; disable/enable (Undo), logout, reset password (link or temporary via `OneTimeSecret`), reset 2FA.
`/superadmin/certificates`	`PlatformCertificatesPage.tsx`	`/superadmin/certificates` (+ actions), `/stats`, `/events`, `/tenants`	Details dialog with integrity check, revoke/restore, `BulkRevokeDialog` (dry run count first).
`/superadmin/analytics`	`PlatformAnalyticsPage.tsx`	`/superadmin/analytics`, `/report`, `/export`	Charts + PDF/CSV exports.
`/superadmin/security`	`SecurityPage.tsx`	`/superadmin/security/*`, `/tenants`, `/users/:id/logout`	Three tabs (`?tab=activity
`/superadmin/announcements`	`AnnouncementsPage.tsx`	`/superadmin/announcements` (+ actions), `/stats`, `/audience`, `/plans`	Status tabs, editor dialog with live bell preview and reach, send now / schedule / draft. `?new=1` opens the editor (dashboard quick action).
`/superadmin/emails`	`EmailTemplatesPage.tsx`	`/superadmin/emails` (+ `/:type`, `/preview`, `/test`), `/superadmin/smtp` (+ `/verify`)	Template list, HTML editor with toolbar and variable insert, live preview (debounced preview endpoint), unsaved-changes guard when switching templates; SMTP tab. `?tab=` and `?type=`.
`/superadmin/settings`	`SystemSettingsPage.tsx`	`/superadmin/settings`, `/superadmin/uploads`	Tabs Platform / Contact & payments / Banner & SEO / Website (`WebsiteContentPanel`) / My account (reuses `AccountTab`). One form for the flat keys (field names map to setting keys), one unsaved guard, nested field errors mapped back.
`/superadmin/maintenance`	`MaintenancePage.tsx`	`/superadmin/maintenance` (+ jobs, cache, errors, backups), `PATCH /superadmin/settings` (maintenance switch/message)	Version, maintenance mode, health cards, jobs with Run now, backups, error log with expandable stacks.

Developer pages (`pages/dev/`)

`/dev/components` (`ComponentGallery.tsx` + `AppBlocksGallery.tsx`): every shared component with sample data, for checking the design system. Only routed in development builds.

10. Operations

How to check, build, deploy and extend EventoraX, plus known limitations. Values to replace before launch are in handover.md.

Checks (run before every commit)

Check	Command	Pass
Backend types	`cd backend && npx tsc --noEmit`	no output
Frontend types + build	`npm run build` (repo root)	"built in …" (a chunk-size note is fine)
Unused code, frontend	`npx knip` (root)	no output
Unused code, backend	`cd backend && npx knip`	no output

There are no automated tests yet. Behaviour is checked with `backend/api-tests.http` (API) and manual-testing.md (UI). If you add tests, add them to CI.

`npx eslint .` currently reports about 80 findings that predate the clean-up (mostly the React-Compiler hook rules `react-hooks/set-state-in-effect` and `purity`, plus `react-refresh/only-export-components`). It is not part of the checks. Fix gradually; don't silence rules globally.

CI (`.github/workflows/ci.yml`)

On every push and pull request to `main` and `integration/v1`, two jobs on Ubuntu with Node 22:

- **Frontend:** `npm ci`, `npm run build`, `npx knip`.
- **Backend:** `npm ci`, `npx prisma generate` (a dummy `DATABASE_URL` only satisfies the config; no database is used), `npx tsc --noEmit`, `npx knip`.

Building and running in production

```
# 1. Website → dist/
npm ci
npm run build

# 2. API → backend/dist/
cd backend
npm ci
npx prisma generate
npm run build

# 3. Database
npx prisma migrate deploy
npx prisma db seed          # first time only (it resets the seeded plans, see chapter 3)

# 4. Start (from backend/, so relative folders resolve there)
NODE_ENV=production npm start
```

One Node process serves the API, `/uploads` and the built website (`web/spa.ts` finds `../dist`). Put HTTPS in front (Hostinger's SSL, or Nginx/Caddy as a reverse proxy; `trust proxy` is already set). Keep the process alive with the host's Node app manager or `pm2` (`pm2 start dist/index.js --name eventorax` from `backend/`).

Folders that must persist across deploys (back them up): `backend/uploads/`, `backend/private-files/`, `backups/`. If they live elsewhere, set `UPLOAD_DIR`, `PRIVATE_DIR`, `BACKUP_DIR`.

Environment: see `backend/.env.example` and the reference table in handover.md.

Upgrading a running server: Maintenance mode on → Back up now → pull, build, `prisma migrate deploy`, restart → health cards OK → maintenance mode off (the same checklist is on the Maintenance page). Open browser tabs reload once by themselves if their page files changed (`vite:preloadError` in `main.tsx`).

Docker: not set up. The old backend-only Dockerfile was removed during the clean-up because it didn't build the website. A future image should be multi-stage: build the root (`dist/`) and `backend/` (`backend/dist/`), copy both plus `backend/node_modules` (production) and `backend/prisma` , run `prisma migrate deploy` on start, and mount the three persistent folders as volumes.

Recipes

Add an organization endpoint

1. Zod schema + handler in `backend/src/controllers/<area>-controller.ts` ; use `req.db!` (scoped), `ok` / `fail` / `validationFail` , `pagination` for lists, `logActivity` for changes.
2. Route in `backend/src/routes/<area>-routes.ts` with `orgScoped` (+ `canWrite` / `adminOnly`); fixed paths before `/:id` .
3. New activity action → a sentence in `src/lib/activityText.ts` .
4. Request in `backend/api-tests.http` ; row in chapter 4.

Add a table

1. Model in `schema.prisma` with `organizationId` + `@@index([organizationId])` + `@@map("snake_case_plural")` .
2. Add the model name to `ORG_SCOPED_MODELS` in `src/prisma/scopedClient.ts` .
3. `npx prisma migrate dev --name add_<thing>` . Commit the migration folder.
4. Update chapter 6.

Add a dashboard page

1. Page in `src/pages/dashboard/<area>/<Name>Page.tsx` following the list or form template.
2. `lazy()` import + `<Route>` in `App.tsx` (wrap with `writer()` or `ProtectedRoute roles` as needed).
3. Menu item in `src/components/layout/nav.ts` (with `need` if restricted).
4. Types in `src/types/<area>.ts` ; statuses in `src/lib/status.ts` .
5. Update the product guide (with a screenshot), chapter 9 and `manual-testing.md` .

Add a system email, a setting, a job

- Email: see chapter 3 ("Adding a new system email").
- Setting: default in `backend/src/prisma/seed.ts` , validator in `controllers/superadmin/settings-controller.ts` (add to the public `/platform` output only if pages need it), field in `SystemSettingsPage.tsx` , type in `src/types/platform.ts` .
- Job: chapter 5 ("Adding a job").

Add a certificate layout

Add it to `LAYOUTS` and draw its decoration in `pdf/certificate.ts` (follow an existing layout), add a thumbnail drawing in `components/certificates/TemplatePickerDialog.tsx` . Keys are `<layout>-<variant>` , so it gets all colours automatically.

Troubleshooting

Symptom	Cause / fix
"Cannot reach the server. Is the backend running?"	Backend not started, or crashed (check its terminal).
504 "Outdated Optimize Dep" / in-app 404 after `npm install`	Restart `npm run dev` (Vite pre-bundles dependencies at start).
`JWT_SECRET must be set … at least 32 characters`	Fill `JWT_SECRET` in `backend/.env` .
Everyone logged out after a deploy	`JWT_SECRET` changed.
"The saved SMTP password can't be decrypted" in logs; 2FA codes rejected	`ENCRYPTION_KEY` (or `JWT_SECRET` without it) changed; re-save SMTP, reset 2FA for users.
Emails don't arrive	Maintenance page → Email card / Email templates → SMTP → Test connection. Without SMTP, emails are printed as `DEV EMAIL` in the backend terminal. Gmail: app password + ~500/day limit.
`402 ORG_EXPIRED`	The organization's `subscriptionEndsAt` passed; confirm a payment or extend in Superadmin → Tenants.

Symptom	Cause / fix
`503 MAINTENANCE` for everyone but the superadmin	Maintenance mode is on (Superadmin → Maintenance).
`429 RATE_LIMITED`	A limiter in `utils/rateLimits.ts` or a route file; limits are per IP (per token for `/ext`).
Camera doesn't start on a phone	The page must be `https://` (or localhost).
Link previews show generic text	`dist/` not built, or `FRONTEND_URL` not the public https URL in production.
"Back up now" fails	`mysqldump` not found: set `MYSQLDUMP_PATH`.
MySQL "Public Key Retrieval is not allowed"	Already handled by `allowPublicKeyRetrieval` in `prisma/client.ts`; on SSL connections it can be removed.
Windows: renaming a file only by case doesn't register	`git mv A tmp && git mv tmp a`.

Known limitations and ideas

- **Background jobs run in the API process.** With several processes each would run every job (announcements are safe thanks to the claim, reminders/summaries mostly safe thanks to marker rows, but not guaranteed). Use one process, or move jobs to a single worker.
- **Seat limits aren't atomic.** Two registrations at the same instant for the last seat can both succeed (check then insert). Acceptable for campus events; a transaction with a row lock would fix it.
- **Logging out** only forgets the token in the browser; the JWT stays valid until it expires. "Log out everywhere" (or a password change) invalidates all of them.
- **Contact messages** are stored in `contact_messages` but there's no inbox page; they reach the team only by email.
- `prisma db seed` **resets the two seeded plans** on every run.
- **Hybrid events:** the ticket shows the venue; only fully online events print the meeting link on the ticket.
- **Expired organizations** see a generic "plan expired" error on locked pages (e.g. Analytics) rather than a dedicated renewal screen.
- **Gmail SMTP** is limited to about 500 emails a day; use a transactional email service in production.
- **Terms and Privacy** are a draft that needs legal review.
- **ESLint** findings (see Checks).